

Computational Workflows for Engineers

Table of contents

Welcome	9
Who This Book Is For	9
How This Book Is Organized	10
What You Won't Find Here	10
Acknowledgments	11
License	11
Introduction	12
What Is a Data Product?	12
The Problem with “Just Code”	13
Why These Tools?	13
Learning by Doing	14
The Passenger and the Driver	14
What You'll Build	15
Prerequisites	16
Before You Begin	16
Foundations Module	16
GitHub Account	16
Package Manager (Homebrew or WinGet)	17
Git	17
Zed	18
Windows Terminal (Windows Only)	18
Database Module	18
DuckDB	18
Programming Module	19
Python (via uv)	19
Integration Module	19
Quarto	19
Verification Checklist	20
Foundations Module	20
Data Engineering Module	20
Python Module	20
Integration Module	20
Getting Help	21

I	Foundations	22
	What You'll Learn	23
	Why This Order?	24
	The Passenger-to-Driver Transformation	24
	Prerequisites	24
	How to Learn This Material	25
1	Computer Fundamentals	26
1.1	Programs and Applications	26
1.2	Memory and Storage	27
1.2.1	When Memory Fills Up	28
1.2.2	Observing What's Running	28
1.3	Files: On Disk and In Memory	28
1.3.1	The Two Lives of a File	29
1.3.2	Why This Matters: The Save Operation	29
1.3.3	The Autosave Generation	29
1.3.4	The In-Memory Editing Model	30
1.4	Text Editors	30
1.4.1	Using Zed	31
1.5	Package Managers	31
1.6	Summary	32
1.7	Glossary	32
1.8	Exercises	33
1.8.1	Question 1.1	33
1.8.2	Question 1.2	34
1.8.3	Question 1.3	34
1.8.4	Question 1.4	34
1.8.5	Question 1.5	34
1.8.6	Question 1.6	35
1.8.7	Question 1.7	35
1.8.8	Question 1.8	35
1.8.9	Question 1.9	36
1.8.10	Question 1.10	36
2	Files and the File System	37
2.1	What Is a File?	37
2.1.1	Text Files	38
2.1.2	Binary Files	39
2.1.3	Why This Matters for Data Work	39
2.2	Common Data File Formats	40
2.2.1	CSV: The Universal Exchange Format	40
2.2.2	JSON: Structured Data as Text	41
2.2.3	YAML and TOML: Configuration Files	41

2.2.4	Markdown: Documentation as Text	42
2.2.5	Excel: The Enterprise Reality	43
2.2.6	Parquet: The Analytics Format	43
2.3	Directories and File Organization	43
2.3.1	The Root Directory	43
2.3.2	Home Directory	45
2.3.3	Visible and Hidden Files	45
2.4	Understanding Paths	46
2.4.1	Absolute Paths	46
2.4.2	Relative Paths	46
2.4.3	Path Separators	46
2.4.4	Special Path Symbols	47
2.4.5	Path Best Practices	47
2.5	Project Organization	47
2.6	Summary	48
2.7	Glossary	49
2.8	Exercises	49
2.8.1	Question 2.1	49
2.8.2	Question 2.2	50
2.8.3	Question 2.3	50
2.8.4	Question 2.4	50
2.8.5	Question 2.5	51
2.8.6	Question 2.6	51
2.8.7	Question 2.7	51
2.8.8	Question 2.8	52
2.8.9	Question 2.9	52
2.8.10	Question 2.10	52
3	The Command Line	53
3.1	Why Learn the Command Line?	53
3.2	The Layers of Your Computing Environment	54
3.3	Setting Up Your Terminal	56
3.3.1	macOS Users	56
3.3.2	Windows Users	56
3.3.3	The Prompt	57
3.4	Navigating the Filesystem	57
3.4.1	Where Am I? The <code>pwd</code> Command	58
3.5	macOS/Linux	58
3.6	Windows	58
3.6.1	What's Here? The <code>ls</code> Command	58
3.7	macOS/Linux	59
3.8	Windows	59
3.8.1	Moving Around: The <code>cd</code> Command	60

3.8.2	Tab Completion: Your Best Friend	61
3.9	Working with Files and Directories	62
3.9.1	Creating Directories: <code>mkdir</code>	62
3.9.2	Removing Files and Directories: <code>rm</code> and <code>rmdir</code>	62
3.10	Understanding Paths	63
3.11	Commands, Arguments, and Options	64
3.12	Types of Commands	65
3.13	Getting Help	65
3.14	A Taste of What's Coming	66
3.15	Summary	66
3.16	Glossary	67
3.17	Exercises	68
3.17.1	Question 3.1	68
3.17.2	Question 3.2	68
3.17.3	Question 3.3	68
3.17.4	Question 3.4	69
3.17.5	Question 3.5	69
3.17.6	Question 3.6	69
3.17.7	Question 3.7	70
3.17.8	Question 3.8	70
3.17.9	Question 3.9	70
3.17.10	Question 3.10	70
4	Version Control with Git	72
4.1	Why Not Just Use Google Docs?	72
4.2	The Git Mental Model	73
4.2.1	The Three States	73
4.2.2	Commits: The Building Blocks	74
4.3	Your First Git Project	75
4.3.1	Initializing a Repository	75
4.3.2	Checking Repository Status	75
4.3.3	Creating and Tracking Files	76
4.3.4	Staging Changes	77
4.3.5	Making Your First Commit	78
4.3.6	The Edit-Stage-Commit Cycle	79
4.3.7	Comparing to Google Docs	80
4.4	Viewing Project History	81
4.5	Undoing Mistakes	82
4.5.1	Discarding Unsaved Edits (Before Staging)	82
4.5.2	Unstaging Changes	83
4.5.3	Amending the Last Commit	83
4.5.4	Reverting a Commit	84

4.6	Branching: Parallel Universes for Your Project	84
4.6.1	The Google Docs Comparison	84
4.6.2	Creating and Switching Branches	84
4.6.3	Working on a Branch	85
4.6.4	Viewing Branches	85
4.6.5	Switching Back and Seeing the Difference	86
4.6.6	Merging Branches	86
4.6.7	Deleting Merged Branches	86
4.7	The .gitignore File	87
4.8	Best Practices for Commits	87
4.8.1	Commit Messages	87
4.8.2	Commit Size	88
4.8.3	Commit Frequency	88
4.8.4	Save Before Committing	88
4.9	Zed and Git: A Quick Reference	88
4.10	Summary	89
4.11	Glossary	89
4.12	Resources	89
5	Exercises	94
5.0.1	Question 4.1	94
5.0.2	Question 4.2	94
5.0.3	Question 4.3	94
5.0.4	Question 4.4	95
5.0.5	Question 4.5	95
5.0.6	Question 4.6	95
5.0.7	Question 4.7	95
5.0.8	Question 4.8	96
5.0.9	Question 4.9	96
5.0.10	Question 4.10	96
6	Collaboration with GitHub	97
6.1	Git is Distributed	97
6.2	Remote Repositories	97
6.2.1	Cloning an Existing Repository	98
6.2.2	Connecting a Local Repository to GitHub	98
6.3	Pushing and Pulling	99
6.3.1	Pushing Changes	99
6.3.2	Pulling Changes	99
6.3.3	Handling Conflicts	100
6.4	The Collaborative Workflow	100
6.4.1	Feature Branch Workflow	100
6.4.2	Pull Requests	101

6.4.3	Code Review	101
6.4.4	Merging Pull Requests	101
6.5	Project Management on GitHub	102
6.5.1	Issues	102
6.5.2	Labels and Milestones	102
6.5.3	GitHub Projects	103
6.5.4	Discussions	103
6.6	Forking: Contributing to Others' Projects	103
6.7	GitHub CLI	104
6.8	Repository Organization	104
6.8.1	README Files	105
6.8.2	License	105
6.8.3	Contributing Guidelines	105
6.8.4	Directory Structure	105
6.9	Security Considerations	106
6.10	Summary	106
6.11	Glossary	107
6.12	Resources	107
6.13	Exercises	108
6.13.1	Question 5.1	108
6.13.2	Question 5.2	108
6.13.3	Question 5.3	109
6.13.4	Question 5.4	109
6.13.5	Question 5.5	109
6.13.6	Question 5.6	110
6.13.7	Question 5.7	110
6.13.8	Question 5.8	110
6.13.9	Question 5.9	111
6.13.10	Question 5.10	111

Appendices 112

A Homebrew 112

A.1	What Problem Does Homebrew Solve?	112
A.2	Why This Matters for Your Workflow	112
A.3	Installation Instructions	113
A.4	What Happens Next	113

B Git Configuration 114

B.1	Configuration Levels	114
B.2	Checking Your Configuration	114
B.3	Required: Your Identity	115

B.4	Verifying Installation	115
B.4.1	Installing Git	116
B.5	Default Branch Name	116
B.6	Line Endings	116
B.7	Default Editor	117
B.8	Useful Aliases	117
B.9	Modern Authentication	118
B.10	Viewing Your Configuration File	118
B.11	Recommended Starter Configuration	119
B.12	Summary	119
B.13	Glossary	119
B.14	Resources	120
C	GitHub Configuration	121
C.1	Creating a GitHub Account	121
C.2	Authentication Methods	121
C.2.1	Recommended: GitHub CLI Authentication	121
C.3	Configuring Your GitHub Profile	122
C.4	GitHub CLI Configuration	123
C.5	Essential GitHub CLI Commands	123
C.6	Notifications and Watching	124
C.7	Connecting Git and GitHub	124
C.8	Troubleshooting Authentication	124
C.9	Summary	125
C.10	Glossary	125
C.11	Resources	125
D	WinGet	128
D.1	What Problem Does WinGet Solve?	128
D.2	Why This Matters for Your Workflow	128
D.3	Checking If WinGet Is Already Installed	129
D.4	Installation Instructions (If Needed)	129
D.5	A Note on Permissions	129
D.6	What Happens Next	129
E	Zed (WIP)	131

Welcome

This is the website for “**Computational Workflows for Engineers**”. This book will teach you how to build data products using SQL, Python, and the professional tools that connect them. You’ll learn to extract data from various sources, transform it into useful forms, create reports and analyses, and share your results through packaging and automation.

In this book, you will find a practicum of skills for engineering data work. Just as a machinist learns to read blueprints and maintain equipment before cutting metal, you’ll learn to navigate file systems and use version control before writing analysis code. These foundational skills aren’t prerequisites to rush through. They’re the professional practices that separate engineers who can reliably build and maintain data systems from those who produce fragile, one-off scripts. You’ll learn to think in workflows: how data moves from source to insight, how code moves from your laptop to production, and how projects move from idea to deliverable. Along the way, you’ll develop the mental models that make debugging systematic rather than desperate, and the habits that make collaboration natural rather than painful.

This book exists because there’s a gap between “knowing how to code” and “knowing how to build things with code.” Many engineers can write a Python script that works once on their machine. Fewer can build a data pipeline that runs reliably, produces traceable results, and can be maintained by someone else six months later. This book bridges that gap by teaching not just the *what* of data engineering tools, but the *why* behind professional practices. When you understand why version control matters, you use it consistently. When you understand why SQL and Python serve different purposes, you choose the right tool for each job. When you understand what makes a data product different from a one-time analysis, you build things that last.

Who This Book Is For

This book is designed for engineering students and professionals who need to work with data but aren’t pursuing careers as software developers. You might be an industrial engineer optimizing manufacturing processes, a civil engineer analyzing infrastructure data, a mechanical engineer processing sensor readings, or any engineer whose work increasingly involves computational analysis.

The book assumes no prior programming experience. If you’ve never opened a terminal or written a line of code, you’ll find everything you need here. If you have some programming

background, you'll likely discover gaps in your workflow knowledge, the professional practices that experienced engineers take for granted but tutorials rarely teach.

How This Book Is Organized

The book progresses through five modules, each building on the previous:

Foundations covers the computational environment: how computers store and process data, how to navigate using the command line, and how to track your work with Git and GitHub. These skills underpin everything else.

Databases introduces databases and the query language that powers analytical work. You'll learn to think in sets rather than loops, and to let the database do the heavy lifting.

Programming teaches programming as a tool for orchestrating workflows. Rather than treating Python as a general-purpose language, we focus on the patterns most useful for data work: reading files, calling APIs, transforming data, and generating outputs.

Integration brings SQL and Python together, showing how to build complete data pipelines that extract, transform, and load data reliably.

Delivery covers the skills that turn projects into products: documentation, testing, packaging, and deployment.

What You Won't Find Here

This is not a data science book. We won't cover statistical modeling, machine learning, or experimental design. Those topics deserve their own treatment, and many excellent resources exist for them.

This is not a software engineering book. We won't build web applications, design APIs, or discuss system architecture at scale. Our focus is narrower: the data pipelines and analytical workflows that engineers build to support their domain work.

This is not a comprehensive reference for any single tool. Python has hundreds of libraries; we'll use a handful. SQL has dozens of dialects; we'll focus on patterns that transfer. The goal is working fluency, not encyclopedic knowledge.

Acknowledgments

This book draws on ideas from David Perkins' *Making Learning Whole*, which shaped the pedagogical approach of learning through complete, meaningful projects rather than isolated skill drills.

License

This website is free to use and is licensed under the [Creative Commons Attribution-NonCommercial-NoDerivs 4.0](#) License. The code examples are licensed under the [MIT License](#).

Introduction

Every piece of data you encounter in your engineering career started somewhere and is going somewhere. A sensor reading captured at a manufacturing station will travel through databases, transformations, and reports before influencing a production decision. A survey response will join thousands of others, get cleaned and aggregated, and eventually appear in a dashboard that shapes policy. Understanding this journey, and knowing how to build the infrastructure that makes it reliable, is what this book is about.

What Is a Data Product?

A **data product** is any system that takes data as input and produces value as output. This definition is deliberately broad. A data product might be a weekly report that summarizes production quality metrics. It might be a dashboard that updates in real-time as new readings arrive. It might be a trained machine learning model that predicts equipment failures. It might be a cleaned and documented dataset that other analysts can trust and reuse.

What unites these examples is that they're not one-time analyses. Someone ran a script once and got an answer. Data products are *systems* designed to produce value repeatedly and reliably. They have inputs that might change. They have logic that transforms those inputs. They have outputs that someone depends on. And crucially, they have lifecycles: they need to be maintained, updated, and eventually retired.

This distinction between a one-time analysis and a data product is the central theme of this book. Anyone can write a script that works once on their laptop with today's data. Building something that works next week, with new data, run by someone else, on a different machine, requires a different set of skills. Those skills are what we'll develop here.

Consider the difference through an analogy from manufacturing. A machinist can create a single prototype part by hand, adjusting as they go, using craft knowledge to achieve the desired result. But manufacturing that part at scale requires something different: documented processes, quality controls, standardized tooling, and systems that work reliably without constant human intervention. The prototype might be beautiful, but it's not a product. The skills this book teaches are the engineering equivalent of moving from prototype to production.

The Problem with “Just Code”

Many introductions to programming treat coding as the core skill. Learn the syntax, understand the concepts, write programs that produce correct output. This approach works for learning to program, but it fails to prepare you for building data products.

In professional data work, the code is often the easy part. Getting data from where it lives into your program is hard. Ensuring your transformations are correct when the input changes is hard. Making your work reproducible so a colleague (or your future self) can understand and modify it is hard. Deploying your solution so it runs reliably without your intervention is hard. Communicating your results so stakeholders can act on them is hard.

The code itself, the Python script or SQL query that performs the transformation, might be twenty lines long. But the infrastructure around that code, the version control, the data connections, the error handling, the documentation, the automation, is what determines whether you’ve built a prototype or a product.

This book teaches both the code and the infrastructure. We’ll certainly write Python and SQL. But we’ll spend equal time on the professional practices that make code useful: organizing projects so they’re navigable, tracking changes so they’re reversible, documenting decisions so they’re understandable, and automating execution so results are reproducible.

Why These Tools?

This book focuses on a specific toolkit: the command line for navigation and automation, Git for version control, SQL for data manipulation, Python for orchestration and transformation, and Quarto for documentation and reporting. This is a deliberate choice, not an accident of the author’s preferences.

SQL is the language of data. Every major database speaks it. Every data warehouse, every analytics platform, every business intelligence tool understands SQL queries. When you need to extract, filter, aggregate, or join data, SQL is almost always the right tool. It’s declarative (you describe *what* you want, not *how* to compute it), which makes it concise and lets the database optimize execution. Learning SQL gives you access to decades of engineering investment in making data operations fast and reliable.

Python is the connective tissue. It’s not the best language for any single task, but it’s good enough at everything and excellent at connecting things together. Python can read files in any format, call any API, invoke any command-line tool, and generate any output. When you need to orchestrate a workflow that touches multiple systems, Python is the natural choice. Its ecosystem of libraries for data work (pandas, polars, DuckDB, httpx) is unmatched.

The command line is the universal interface. Every tool you’ll encounter, from Git to Python to database clients to cloud services, can be operated through text commands. Learning the

command line gives you access to every tool, including ones that haven't been invented yet. It also enables automation: anything you can type, you can script.

Git solves the problem that every engineer eventually encounters: “I made changes and now everything is broken, how do I get back to the version that worked?” But Git is much more than an undo system. It's a collaboration protocol, a deployment mechanism, and a documentation tool. Professional software development runs on Git, and data engineering is increasingly adopting the same practices.

These tools share a philosophy: they're composable, scriptable, and open. You can combine them in ways their creators never imagined. You can automate anything you can do manually. And you can inspect and understand what they're doing, rather than trusting a black box.

Learning by Doing

This book follows a pedagogical approach called “learning by wholes,” developed by David Perkins in *Making Learning Whole*. The core idea is that learners should engage with complete, meaningful activities from the beginning, rather than accumulating isolated skills that only make sense later.

In practice, this means we won't spend weeks on Python syntax before writing useful programs. Instead, you'll build small but complete data products from early in the book. Each project will be a “junior version of the game,” simplified enough to be achievable but complete enough to be meaningful. As you progress, the projects become more sophisticated, but the basic shape remains: data comes in, transformations happen, value goes out.

This approach requires a different relationship with confusion. When you're playing the whole game, you'll encounter concepts before you fully understand them. You'll use tools before you've mastered them. This is intentional. Understanding often follows use, not the other way around. Trust the process, and return to earlier material as later chapters illuminate what you learned before.

The alternative, mastering each component in isolation before combining them, produces a different kind of confusion: the confusion of not knowing why any of this matters. Students who learn syntax without context struggle to apply their knowledge to real problems. Students who build complete projects, even imperfect ones, develop intuition that transfers.

The Passenger and the Driver

There's a metaphor we'll return to throughout this book: the difference between a *passenger* and a *driver*.

A passenger uses a computer by clicking buttons and hoping for the best. When something goes wrong, they restart and try again. When they need to accomplish a task, they search for a tutorial with step-by-step instructions. They can get things done, but they're dependent on tools behaving exactly as expected and tutorials existing for exactly their situation.

A driver understands what's happening under the interface. When something goes wrong, they can reason about the cause. When they need to accomplish a new task, they can combine familiar tools in new ways. They're not dependent on tutorials because they have mental models that generate solutions.

This book aims to transform you from passenger to driver. That transformation requires more than learning commands and syntax. It requires building mental models: understanding why files are organized into directories, why version control requires explicit commits, why SQL and Python serve different purposes. These mental models are what let you adapt when circumstances change, debug when things break, and create when no tutorial exists.

The transformation isn't instant. You'll start as a passenger in each new domain, following instructions without full understanding. That's fine. Understanding develops through use. But the goal is always to reach the point where you could teach someone else, where you understand not just *what* to do but *why* it works.

What You'll Build

By the end of this book, you'll have built several complete data products:

You'll create automated reports that pull data from sources, perform transformations, and generate formatted output, all triggered by a single command or scheduled to run automatically.

You'll build data pipelines that extract information from files and APIs, clean and transform it using SQL, and load it into databases where it can be queried and analyzed.

You'll develop documentation that explains not just what your code does, but why it's designed that way, rendered into professional formats that stakeholders can read.

And you'll assemble a portfolio of projects tracked in Git, hosted on GitHub, demonstrating your ability to build maintainable, reproducible data systems.

These aren't hypothetical exercises. They're the actual deliverables that data-focused engineering roles require. The skills you develop building them are the skills employers are looking for.

Prerequisites

This chapter lists the tools you'll need throughout the book. Rather than installing everything at once, you'll set up each tool as it becomes relevant. This approach lets you start quickly and spreads the installation work across the course.

Each tool listing includes what it does, when you'll need it, and a reference to installation instructions. If you encounter problems during installation, check the appendix for troubleshooting steps specific to your operating system.

Before You Begin

You'll need a computer running Windows 11, macOS, or Linux. Most tools work across all platforms, though installation steps differ. The book provides instructions for both Windows and macOS; Linux users can generally follow the macOS instructions with minor adjustments.

You should have administrator access to install software. If you're using a managed computer (such as a university lab machine), verify that you can install programs or contact your IT department for assistance.

A stable internet connection is required for downloading tools and, later, for pushing code to GitHub and accessing online data sources.

Foundations Module

These tools support should be installed before or during the first week of work.

GitHub Account

What it is: A free account on github.com, the platform for hosting Git repositories and collaborating on projects.

When you need it: Chapter 6 (Collaboration with GitHub), though creating the account early is recommended.

How to get it: Visit github.com and sign up with your email address. Choose a professional username since this account may become part of your portfolio. Enable two-factor authentication for security.

Notes: Use your personal email rather than a school email, so you retain access after graduation. GitHub offers free features that are sufficient for this course; you don't need a paid plan.

Package Manager (Homebrew or WinGet)

What it is: A tool that automates installing, updating, and managing software from the command line.

When you need it: Immediately, before installing other tools.

How to get it:

- **Windows:** WinGet comes pre-installed on Windows 11 and recent Windows 10 versions. Open Windows Terminal and type `winget --version` to verify it's available.
- **macOS:** Install Homebrew by following instructions at brew.sh. This requires running a command in Terminal.

Notes: While technically optional (you can install everything manually), a package manager makes installation and updates dramatically easier. The small upfront investment pays off immediately.

Git

What it is: A version control system that tracks changes to files and enables collaboration.

When you need it: Chapter 4 (Version Control with Git).

How to get it:

- **Windows:** `winget install Git.Git`
- **macOS:** `brew install git`

After installation, configure your identity:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Notes: Use the same email address you used for your GitHub account. This links your commits to your GitHub profile.

Zed

What it is: A modern text editor designed for programming, with features like syntax highlighting, auto-completion, and an integrated terminal.

When you need it: Chapter 1 (Computer Fundamentals) and throughout the book.

How to get it:

- **Windows:** `winget install Zed.Zed`
- **macOS:** `brew install --cask zed`

Alternatively, download from zed.dev.

Notes: Other text editors (VS Code, Sublime Text, etc.) work fine if you have a strong preference. The book uses Zed for examples because of its clean interface and excellent Git integration, but the concepts transfer to any editor.

Windows Terminal (Windows Only)

What it is: Microsoft's modern terminal application, providing a clean interface for command-line work.

When you need it: Chapter 3 (The Command Line).

How to get it: Install from the [Microsoft Store](https://microsoft.com/store) or via `winget install Microsoft.WindowsTerminal`.

Notes: Windows Terminal should be your default terminal application for this course. Avoid using the legacy Command Prompt or standalone PowerShell applications.

Database Module

These tools support the SQL and database chapters.

DuckDB

What it is: An embedded analytical database that runs SQL queries on local files without server setup.

When you need it: The SQL chapters (to be referenced once written).

How to get it:

- **Windows:** `winget install DuckDB.cli`
- **macOS:** `brew install duckdb`

Notes: DuckDB can query CSV, Parquet, and JSON files directly, making it ideal for learning SQL without database administration overhead.

Programming Module

These tools support the Python chapters.

Python (via uv)

What it is: The Python programming language, installed and managed through the `uv` tool for reliable dependency management.

When you need it: The Python chapters (to be referenced once written).

How to get it:

- **Windows:** `winget install astral-sh.uv`
- **macOS:** `brew install uv`

Then install Python: `uv python install 3.13`

Notes: We use `uv` rather than installing Python directly because `uv` handles virtual environments and dependencies more reliably. This avoids the “it works on my machine” problems that plague Python development.

Integration Module

These tools support the chapters on combining SQL, Python, and documentation.

Quarto

What it is: A publishing system that renders Markdown documents into HTML, PDF, presentations, and other formats.

When you need it: The documentation and reporting chapters (to be referenced once written).

How to get it:

- **Windows:** `winget install Posit.Quarto`
- **macOS:** `brew install --cask quarto`

Notes: Quarto is what renders this book. Learning it gives you the ability to create professional documentation for your own projects.

Verification Checklist

After installing tools for each module, verify they're working:

Foundations Module

```
git --version      # Should print version number
zed --version      # Should print version number (or open Zed)
```

Data Engineering Module

```
duckdb --version   # Should print version number
```

Python Module

```
uv --version        # Should print version number
python --version     # Should print Python 3.13.x (after uv python install)
```

Integration Module

```
quarto --version    # Should print version number
```

If any command fails with “command not found” or similar, revisit the installation instructions or check the troubleshooting appendix.

Getting Help

Installation problems are common and usually solvable. If you encounter issues:

1. Read the error message carefully. It often contains clues about what went wrong.
2. Search the error message online. Someone has likely encountered the same problem.
3. Check the tool's official documentation for platform-specific instructions.
4. Ask for help in class or office hours, bringing the exact error message.

Don't let installation friction discourage you. Setting up a development environment is genuinely tricky, even for experienced developers. Once configured, these tools will serve you throughout the course and beyond.

Part I

Foundations

Before you write a single line of Python or craft your first SQL query, you need to understand the environment where your code will run. This module builds the computational foundation that makes everything else in the book possible.

Most engineering students arrive with years of experience using computers, but that experience is often as a *passenger*. You’ve clicked through interfaces, waited for applications to respond, and restarted when things went wrong. That approach works fine for everyday tasks, but it breaks down when you’re building data products. Building requires understanding what’s actually happening inside the machine, how files are organized and accessed, how tools communicate with each other, and how to track and collaborate on complex projects.

This module transforms you from passenger to *driver*. By the end, you’ll have mental models for how computers store and process data, fluency with the command line as your primary interface for professional work, and a complete version control workflow using Git and GitHub. These aren’t just prerequisites to check off before the “real” content begins. They’re the foundation that professionals use every day, and mastering them early will make everything else you learn more intuitive and effective.

What You’ll Learn

Computer Fundamentals (Chapter 1) establishes the mental models that explain why computers behave the way they do. You’ll understand the critical distinction between memory and storage, why unsaved work can vanish, and what it means for code to exist as plain text. These concepts will resurface constantly as you work with data.

Files and the File System (Chapter 2) explores how computers organize information. You’ll learn to think in terms of paths and directories, understand the difference between text and binary files, and survey the file formats you’ll encounter when building data products. This knowledge is essential for writing code that reads data from one location and writes results to another.

The Command Line (Chapter 3) introduces the text-based interface that connects all your tools. While graphical interfaces hide complexity behind buttons and menus, the command line gives you direct access to your computer’s capabilities. You’ll learn the core commands for navigation and file manipulation, building vocabulary that transfers to every other tool in the book.

Version Control with Git (Chapter 4) solves a problem every engineer faces: tracking changes to a project over time. Git provides intentional checkpoints, the ability to experiment without risk, and a complete history of how your work evolved. You’ll learn to think in commits and branches, a mental model that fundamentally changes how you approach project work.

Collaboration with GitHub (Chapter 6) extends version control to team settings. GitHub hosts your repositories in the cloud, enables code review, and provides project management tools.

The workflows you learn here, pushing, pulling, branching, and merging, are how professional engineering teams coordinate their efforts.

Why This Order?

Each chapter builds on the previous one. Understanding memory and storage explains why you must save files before Git can see them. Understanding the file system explains the paths you'll type on the command line. Command line fluency is required to run Git commands. And Git mastery is the prerequisite for GitHub collaboration.

This progression mirrors how you'll actually work. When you eventually write a Python script that processes data files and commits the results to a Git repository, you'll be using concepts from every chapter in this module. The foundations aren't separate from "real" programming. They're woven into every task you'll perform.

The Passenger-to-Driver Transformation

There's a difference between knowing *that* something works and understanding *why* it works. Passengers know that clicking "Save" preserves their document. Drivers understand that clicking "Save" writes the in-memory representation to disk, replacing the previous version. This deeper understanding matters because it lets you reason about problems when things go wrong, and things always go wrong eventually.

Throughout this module, we'll build these deeper mental models. When you encounter a "file not found" error later in the book, you'll understand it as a path problem, not a mysterious failure. When a Git merge conflict appears, you'll recognize it as a natural consequence of parallel development, not a catastrophe. This understanding is what separates professionals from perpetual beginners.

Prerequisites

This module assumes you can use a computer for everyday tasks: browsing the web, creating documents, installing applications. If you've never opened a terminal or typed a command, that's perfectly fine. We'll start from the beginning.

You will need a computer (Windows, macOS, or Linux) where you can install software. Some tools require administrator access to install, so if you're using a managed machine, ensure you have the necessary permissions or contact your IT department.

How to Learn This Material

These chapters are designed to be read actively. When you see a command, type it yourself rather than copying and pasting. When you see a concept explained, pause and make sure you can explain it back in your own words. The command line in particular rewards muscle memory. The more you type, the faster the commands become automatic.

Don't rush through this module to get to the “interesting” parts. The hours you invest here will pay dividends throughout the rest of the book and throughout your career. Version control alone will save you from countless disasters, but only if you've internalized the workflow deeply enough to use it consistently.

If you already have experience with some of these topics, skim the familiar sections but pay attention to the mental models being developed. You might know *how* to use Git without having a clear picture of *why* the staging area exists. Those conceptual gaps tend to cause problems later.

Let's begin.

1 Computer Fundamentals

Before you write your first line of code or type your first command, you need a mental model of what’s actually happening inside your computer. This isn’t about memorizing specifications or understanding circuit design. It’s about developing intuition for *why* computers behave the way they do, so that when something goes wrong, you can reason about the problem instead of flailing randomly.

Most people interact with computers as passengers. They click buttons, wait for responses, and hope for the best. When something breaks, they restart and pray. This works fine for casual use, but it’s insufficient for building data products. You need to become a driver, someone who understands the machine well enough to diagnose problems and make informed decisions about how to structure your work.

1.1 Programs and Applications

The words “program” and “application” get tossed around interchangeably in casual conversation. For working professionals who build software, the distinction matters.

A **program** is the most fundamental unit of executable software: a set of instructions written for a computer to execute. That’s the entire definition. Notice what it doesn’t promise. There’s no guarantee of usability, no requirement for visual interfaces, no assumption that a human will ever interact with it directly. A Python script that renames files is a program. A scheduled job that runs at midnight to clean up old logs is a program. A command-line tool you invoke, watch complete, and forget is a program. If it runs on a computer, it’s probably a program.

An **application** is a specific kind of program with a higher purpose: a program designed to be used directly by an end user to accomplish a meaningful task. Applications are built for humans, not just machines. They have a user interface of some kind. They handle errors gracefully instead of crashing with cryptic messages. They persist settings between sessions. A desktop photo editor is an application. Gmail is an application. Polished command-line tools like `git` and `quarto` are applications too, because of the care taken in their design: comprehensive help systems, consistent command structures, meaningful error messages.

An application makes a promise: you can hand this to someone who didn’t write it, and they won’t immediately regret the experience.

In this book, you will write programs. Lots of them. Scripts that process data, automate workflows, and transform information. These programs will assume you know what you're doing because you wrote them for yourself. But by the end of the book, some of your programs will start becoming applications. Your final project will need documentation and graceful error handling. It will need to work for someone who isn't you.

For the rest of this book, when I say **program**, I mean executable code that accomplishes a computational task. When I say **application**, I mean software deliberately designed for end-user interaction. When I say **script**, I mean a program written in an interpreted language (like Python) that typically runs from the command line.

1.2 Memory and Storage

Here's a truth that will save you hours of confusion: computers are not slow. Accessing data is slow.

When your code takes forever to run or loading a file makes you wonder if your machine froze, the problem is almost never that your CPU can't do math fast enough. Modern processors execute billions of operations per second. The bottleneck is almost always about where the data lives.

Your computer stores data in two fundamentally different places: **memory** (RAM) and **storage** (your disk, whether SSD or hard drive). Understanding the difference between them is essential.

RAM (Random Access Memory) is fast, temporary workspace. When programs run, they exist in RAM. When you open a file, its contents get copied into RAM so you can work with them. RAM is quick to access, but it has two important limitations: it's relatively small (typically 8-16 GB on a laptop), and it's volatile, meaning everything in RAM disappears when the power goes off.

Storage (disk) is slower, permanent space. Your files live on disk. Your programs are stored on disk. When you save something, you're writing it to disk. Storage is much larger than RAM (hundreds of gigabytes to terabytes) and persists when the power is removed, but accessing it takes dramatically longer, roughly a thousand times slower than accessing RAM.

This explains several patterns you'll encounter. Loading a large file feels slow because that data must travel from disk into RAM before any work can happen. "Just load the whole file" stops working at scale because datasets can exceed available RAM. The same analysis runs fast on a small sample but crawls on the full dataset because the small sample fits comfortably in memory while the full dataset requires constant disk access.

Every performance problem you encounter can be reframed as a question about where the data lives. Once you internalize this, debugging becomes more systematic.

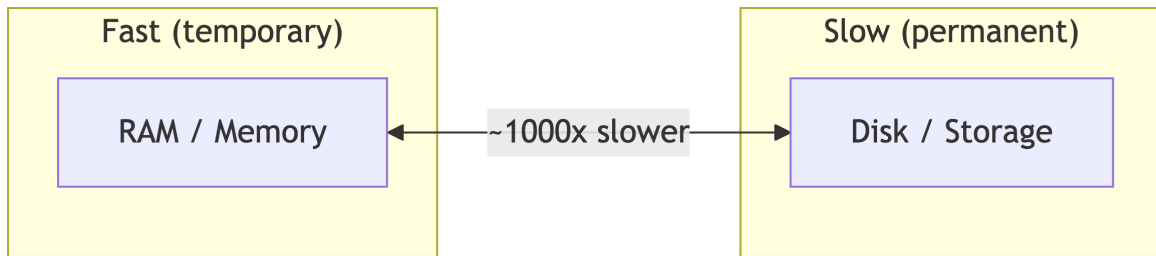


Figure 1.1: RAM is fast but temporary; disk is slow but permanent

1.2.1 When Memory Fills Up

RAM is a finite workspace. Fill it up and your computer has no room to operate. When this happens, your operating system might start using disk space as overflow memory, a technique called “swapping.” This keeps things running but at a severe performance cost. Your program might also crash outright with an “out of memory” error, which is the computer protecting itself by refusing to continue.

A few things students routinely misunderstand: closing a program frees memory (when you quit an application, the operating system reclaims whatever RAM it was using). Restarting fixes “mysterious” issues (many strange behaviors result from accumulated state in memory; restarting clears the slate). Large files don’t shrink when loaded (a 1GB file on disk becomes roughly 1GB of data in memory, often more).

1.2.2 Observing What’s Running

Every operating system provides tools to see what processes are running and how much memory they’re consuming. On Windows, Task Manager (Ctrl+Shift+Esc) shows this information. On macOS, Activity Monitor (in Applications → Utilities) provides similar details.

Learning to use these tools helps you understand what your computer is actually doing. When something runs slowly, check whether a process is consuming all available CPU or memory. When your computer feels sluggish, see what’s running in the background. A dozen browser tabs can consume gigabytes of RAM. The process view reveals activity that graphical interfaces often hide.

1.3 Files: On Disk and In Memory

You’ve been working with files your entire life, but the way files behave when you’re programming differs from what you’re used to with apps like Google Docs or Microsoft Word. Understanding this difference now will prevent significant confusion later.

1.3.1 The Two Lives of a File

Every file you work with has two potential representations: its permanent form on disk and its temporary form in memory when a program opens it.

When a file sits on your disk unopened, it's just a sequence of bytes stored magnetically or electronically. It exists independently of any program. You can copy it, move it, delete it, or leave it alone for years. The file persists whether your computer is on or off.

When you open that file in a program, something important happens: the program reads the bytes from disk and creates a *copy* in memory. This in-memory copy is what you actually see and edit. The original file on disk remains unchanged until you explicitly save.

Think of it like a library book. The book on the shelf (disk) is the permanent copy. When you check it out and start taking notes in a notebook (memory), you're working with your own copy. The library book doesn't change until you somehow transfer your notes back to it.

1.3.2 Why This Matters: The Save Operation

When you click “Save” or press Ctrl+S (Cmd+S on Mac), you're telling the program to write the in-memory version back to disk, replacing the old version. Until you save, your changes exist only in RAM. This has consequences:

If your computer loses power before you save, your changes vanish. They existed only in volatile memory, which clears when power disappears.

If you close the program without saving, your changes vanish. The program discards the in-memory copy.

If you open the same file in two different programs simultaneously, each program has its own in-memory copy. They can diverge, and whoever saves last “wins.”

1.3.3 The Autosave Generation

If you've grown up with Google Docs, iCloud, or OneDrive, this might feel alien. Modern productivity apps continuously sync your work to the cloud. There's no “save” button because saving happens automatically in the background. You've never experienced losing work to an unsaved document because that failure mode has been designed away.

This is genuinely good design for documents. A half-finished paragraph is still a readable paragraph. Constant automatic saving protects users from data loss with essentially no downside.

But code is different. A half-finished function is a syntax error. A partially modified data pipeline produces wrong results or crashes. Code exists in one of two states: working or broken. There's no meaningful in-between.

This is why code editors traditionally require manual saves. The act of saving is a commitment: “This version is complete enough to keep.” You might save frequently while working, but each save is a deliberate choice rather than an automatic background process. You're not just preserving text; you're check-pointing a system that must maintain internal consistency.

1.3.4 The In-Memory Editing Model

Let's trace exactly what happens when you edit a file in a text editor like Zed.

You open `script.py`. Zed reads the file from disk into memory. Your screen now shows the in-memory representation.

You type some code. Each keystroke modifies the in-memory copy. The file on disk hasn't changed at all. If someone else opened the same file right now, they'd see the old version.

You notice Zed shows a dot or indicator next to the filename. This signals unsaved changes, that the in-memory version differs from the disk version.

You press `Cmd+S` (or `Ctrl+S`). Zed writes the in-memory contents to disk, replacing the old file. The unsaved indicator disappears. The disk and memory versions now match.

You close Zed. The in-memory copy is discarded. But that's fine because you saved, so the disk version contains all your work.

This model, where explicit saves checkpoint your work, feeds directly into version control. Git doesn't track every keystroke; it tracks deliberate commits that you choose to make. Understanding that “save” is an intentional act prepares you for understanding that “commit” is an even more intentional act. We'll explore this connection in detail in [Chapter 4](#).

1.4 Text Editors

To write code, you need a text editor. This is different from a word processor like Microsoft Word or Google Docs.

A **word processor** works with rich text: text with formatting attached like bold, italics, fonts, and colors. When you see styled text on screen, you're looking at rich text. Word processors embed formatting codes in your document, codes that are invisible to you but present in the file.

A **text editor** works with plain text: just characters with no formatting information. What you see is exactly what's in the file. No hidden codes, no embedded styles.

Code must be plain text. When Python reads your script, it expects exactly the characters you wrote. Hidden formatting codes would cause syntax errors. This is why you can't write code in Word, even if you turn off all visible formatting, the file format itself embeds information that confuses programming languages.

1.4.1 Using Zed

For this book, we'll use **Zed**, a modern text editor designed for professional programming work. Zed provides the features you need without unnecessary complexity:

Syntax highlighting colors different parts of your code based on their meaning. Keywords appear in one color, strings in another, comments in another. This makes code dramatically easier to read and helps you spot errors.

Auto-completion suggests completions as you type. Start typing a function name and Zed offers to finish it, reducing typos and helping you discover available options.

Integrated terminal embeds a command-line interface directly in the editor. Instead of switching between windows, you can write code and run commands in one place. When I refer to “the command line” in later chapters, you can use Zed's integrated terminal.

Unsaved change indicators show when your in-memory edits differ from the disk version. A dot next to the filename means you have unsaved work.

1.5 Package Managers

When you install an app on your phone, you open the App Store or Google Play, tap a button, and the app appears ready to use. You never think about where the files go or what other software the app might need.

A **package manager** brings this convenience to developer tools on your computer. It handles installing, upgrading, and uninstalling software packages. Rather than visiting websites and clicking through setup wizards, you tell the package manager what you want and it takes care of the rest.

Package managers also handle **dependencies**: other tools and libraries that a piece of software needs to function. If you install a tool that requires three other components, the package manager automatically installs those prerequisites in the correct order.

For **Windows**, the built-in package manager is **WinGet**. It comes pre-installed on Windows 11 and recent versions of Windows 10:

For **macOS**, **Homebrew** has become the standard. After a one-time installation, you can install tools like this:

Listing 1.1 terminal (PowerShell)

```
> winget install Git.Git
```

Listing 1.2 terminal (zsh)

```
$ brew install git
```

Throughout this book, when we introduce new tools, I'll provide package manager commands as the recommended installation method. Getting comfortable with your package manager early makes everything else smoother.

1.6 Summary

This chapter established the mental models you need to reason about computational systems.

Programs are executable instructions; applications are programs designed for end users with thoughtful interfaces and error handling. Most of your bookwork will produce programs; your final project should aspire toward being an application.

Memory (RAM) is fast, temporary workspace where programs run and files are edited. Storage (disk) is slower, permanent space where files live. The distinction between RAM and disk explains why loading large files takes time and why unsaved work can be lost. Every performance problem is ultimately a question about where the data lives.

Files exist in two forms: permanently on disk and temporarily in memory when opened by a program. Editing happens to the in-memory copy; saving writes that copy back to disk. Unlike autosaving productivity apps, code editors require manual saves because code must maintain consistency, a half-finished function is a syntax error. This deliberate checkpoint model prepares you for version control.

Text editors work with plain text; word processors embed hidden formatting. Code must be plain text, which is why you need a proper text editor like Zed. Package managers (WinGet on Windows, Homebrew on macOS) simplify installing and managing the developer tools you'll use throughout this book.

1.7 Glossary

Application A program designed for direct end-user interaction, with a thoughtful interface, error handling, and documentation.

Dependency Software that another piece of software requires to function. Package managers automatically install dependencies.

Disk (Storage) Permanent data storage that persists when power is removed. Slower than RAM but much larger and non-volatile.

Memory (RAM) Random Access Memory; fast, temporary workspace where programs run and data is actively processed. Contents are lost when power is removed.

Package manager A tool that automates installing, updating, and removing software packages, handling dependencies automatically.

Plain text Text without any formatting codes or styling information. What you see is exactly what's in the file. Code must be plain text.

Process A running instance of a program, with its own memory allocation and state.

Program A set of instructions written for a computer to execute. The fundamental unit of executable software.

Rich text Text with formatting information attached (bold, fonts, styles). Word processors work with rich text.

Save The operation of writing the in-memory version of a file back to disk, replacing the previous disk version.

Script A program written in an interpreted language (like Python) that runs through an interpreter rather than being compiled to machine code.

Syntax highlighting A text editor feature that displays different parts of code in different colors based on their grammatical role.

Text editor A program for creating and modifying plain text files. Essential for writing code.

Volatile Memory that loses its contents when power is removed. RAM is volatile; disk storage is not.

Word processor A program for creating documents with rich text formatting, like Microsoft Word. Not suitable for writing code.

1.8 Exercises

1.8.1 Question 1.1

What is the primary difference between a **program** and an **application**?

- A) Programs run faster than applications
- B) Applications are designed for end-user interaction with thoughtful interfaces
- C) Programs can only run on servers
- D) Applications cannot be run from the command line

1.8.2 Question 1.2

Which of the following is TRUE about RAM (memory)?

- A) RAM retains data when the computer is powered off
- B) RAM is slower to access than disk storage
- C) RAM is where programs run and files are actively edited
- D) RAM typically has more capacity than disk storage

1.8.3 Question 1.3

Why does loading a large file feel slow?

- A) The CPU cannot process the file fast enough
- B) The file must be copied from disk into RAM before work can begin
- C) Large files are always compressed and need decompression
- D) The operating system limits file access speed

1.8.4 Question 1.4

What happens to unsaved changes in a text editor if your computer loses power?

- A) They are automatically recovered from the cloud
- B) They are saved to a temporary backup file
- C) They are lost because they only existed in volatile RAM
- D) They are stored in the CPU cache

1.8.5 Question 1.5

Why can't you write code in Microsoft Word?

- A) Word doesn't support typing special characters

- B) Word embeds hidden formatting codes that confuse programming languages
- C) Word files are too large for code
- D) Word cannot save files with .py extensions

1.8.6 Question 1.6

What does “volatile” mean when describing computer memory?

- A) The memory is dangerous and may cause fires
- B) The memory contents are lost when power is removed
- C) The memory cannot store large files
- D) The memory changes unpredictably during use

1.8.7 Question 1.7

Which statement about package managers is TRUE?

- A) Package managers only work on Linux systems
- B) Package managers automatically handle dependencies when installing software
- C) Package managers require an internet connection for all operations
- D) Package managers replace the need for text editors

1.8.8 Question 1.8

A **script** is best described as:

- A) A compiled program that runs directly on hardware
- B) A program written in an interpreted language like Python
- C) A graphical application with a user interface
- D) A document containing formatting instructions

1.8.9 Question 1.9

What does syntax highlighting in a text editor do?

- A) Automatically corrects spelling errors in code
- B) Colors different parts of code based on their grammatical role
- C) Highlights lines that contain errors
- D) Makes all text the same color for consistency

1.8.10 Question 1.10

True or False: Closing a program frees the RAM that program was using.

- A) True
- B) False

2 Files and the File System

Before you can work with the command line, you need to understand the landscape you'll be navigating: your computer's file system. This might seem obvious, you've been saving documents and opening folders for years. But there's a growing disconnect between how modern software presents files and how computers actually organize them, and that gap creates real problems when you start building data products.

A recent study found that many students who grew up with smartphones and search-based interfaces struggle with the fundamental concept of files stored in hierarchical folders. They're used to apps that abstract away storage entirely: photos appear in Photos, documents in Google Docs, music in Spotify. The underlying organization, that everything ultimately lives as files in directories on some storage device, has become invisible.

This invisibility is fine for casual use, but it breaks down quickly in professional computing. When you write a Python script that reads data from one location and writes results to another, you need to specify *exactly* where those locations are. When you organize a project with code, data, documentation, and outputs, you need a mental model of how directories nest inside each other. When something goes wrong, you need to find the actual file and examine it.

This chapter builds that foundation. We'll explore what files actually are, how they're organized into directories, and how you identify locations using paths. We'll also survey the common file formats you'll encounter when building data products, because knowing that a `.csv` is plain text you can read while a `.xlsx` is a compressed archive of XML files changes how you approach both.

2.1 What Is a File?

At the most fundamental level, your computer stores everything as sequences of binary digits: ones and zeros. Every photo, document, program, and dataset ultimately reduces to patterns of electrical charges or magnetic orientations representing these binary values. A **file** is simply a named collection of this binary data stored on a disk or other storage medium.

What makes files useful is *interpretation*. The same sequence of bytes means different things depending on what kind of file it is and what program reads it. This is where file formats come in, agreed-upon conventions that specify how to interpret a file's bytes.

Files fall into two broad categories: **text files** and **binary files**.

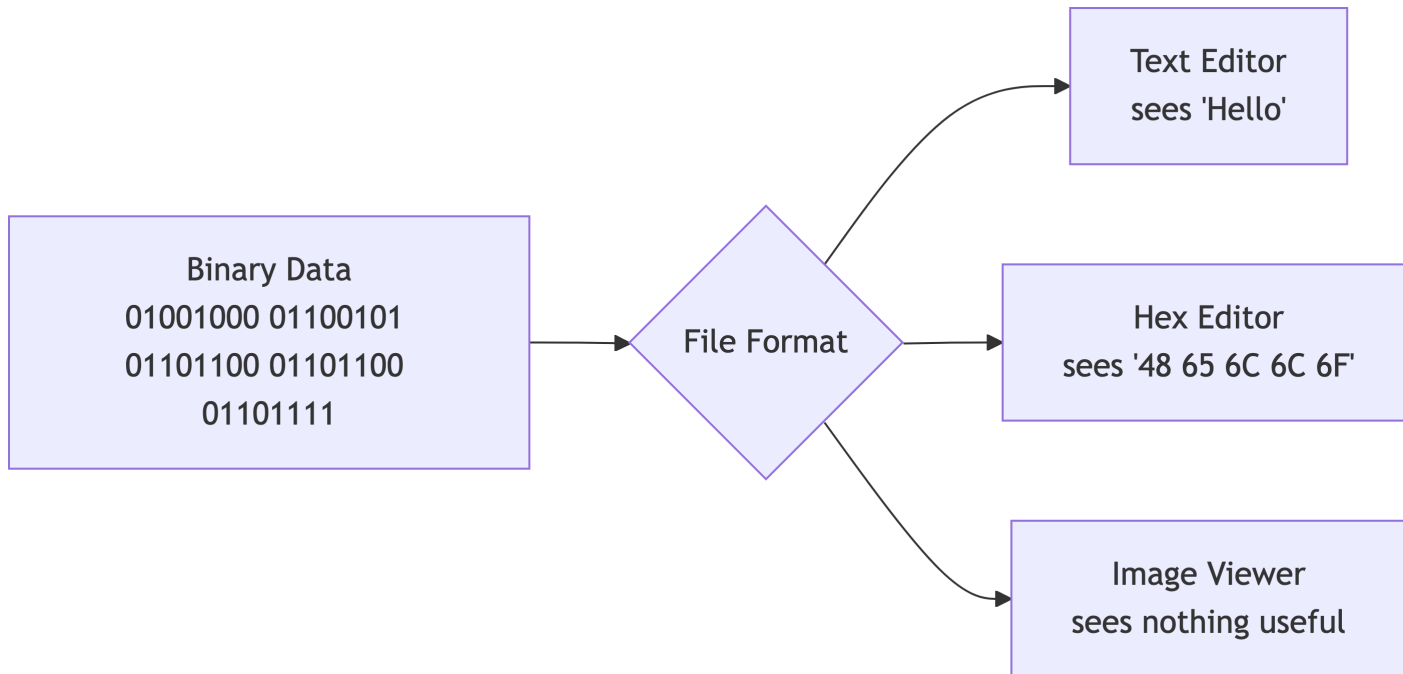


Figure 2.1: Every file is ultimately a sequence of binary data interpreted according to its format

2.1.1 Text Files

Text files store human-readable characters using an encoding standard (typically UTF-8 in modern systems). Each character maps to a specific number, and those numbers are stored as bytes. When you open a text file in any text editor, you can read its contents directly.

Common text file formats include:

- **Plain text** (.txt): Unformatted text with no special structure
- **Markdown** (.md): Text with lightweight formatting syntax (what this book is written in)
- **CSV** (.csv): Comma-separated values, tabular data as plain text
- **JSON** (.json): JavaScript Object Notation, structured data in a human-readable format
- **YAML** (.yaml, .yml): Yet Another Markup Language, configuration files with readable structure
- **TOML** (.toml): Tom's Obvious Minimal Language, another configuration format
- **Source code** (.py, .sql, .r, .js): Programming instructions as text

The beauty of text files is their transparency. You can open any text file in any text editor and see exactly what's inside. This makes them easy to inspect, debug, and version control. When something goes wrong with a CSV file, you can open it in a text editor and see the raw data.

Here's what a CSV file actually looks like inside:

Listing 2.1 products.csv

```
product_id,name,price,quantity
1001,Widget A,29.99,150
1002,Widget B,49.99,75
1003,Gadget X,99.99,200
```

It's just text with commas separating values and newlines separating rows. Any program that understands the CSV convention can read it.

2.1.2 Binary Files

Binary files store data in formats that aren't meant to be human-readable. The bytes represent something other than text characters, perhaps image pixels, compressed data, or complex document structures.

Common binary file formats include:

- **Images** (.jpg, .png, .gif): Pixel data, often compressed
- **Excel spreadsheets** (.xlsx): Actually a ZIP archive containing XML files
- **PDF documents** (.pdf): Portable Document Format, complex layout information
- **Parquet** (.parquet): Columnar data storage optimized for analytics
- **SQLite databases** (.db, .sqlite): Self-contained relational databases
- **Compiled programs** (.exe, .app): Machine code instructions

If you open a binary file in a text editor, you'll see gibberish, random characters, special symbols, and unprintable sequences. This isn't corruption; it's simply that the bytes weren't meant to be interpreted as text.

i File Extensions Are Just Conventions

The .csv or .xlsx extension tells programs (and humans) how to interpret the file, but it's not enforced by the operating system. You could rename a JPEG image to have a .txt extension, it wouldn't change the actual content, just confuse programs trying to open it. When troubleshooting, remember that the extension is a hint, not a guarantee.

2.1.3 Why This Matters for Data Work

Understanding the text/binary distinction has practical implications:

1. **Version control** (Git, which we'll cover in Chapter 4) works excellently with text files because it can track line-by-line changes. Binary files can be stored but changes can't be meaningfully compared.
2. **Debugging** text-based data is straightforward, open the file and look at it. Binary formats require specialized tools.
3. **Interoperability**: Text formats like CSV and JSON can be read by virtually any programming language or tool. Binary formats often require specific libraries.
4. **Size vs. readability tradeoff**: Binary formats like Parquet are more compact and faster to process at scale, but text formats are easier to inspect and share.

For this course, we'll primarily work with text-based formats (CSV, JSON, SQL) and occasionally encounter binary formats when using specialized tools.

2.2 Common Data File Formats

Let's look more closely at the file formats you'll encounter most often when building data products.

2.2.1 CSV: The Universal Exchange Format

CSV (Comma-Separated Values) is the lowest-common-denominator format for tabular data. Nearly every tool can read and write CSV files, making them ideal for data exchange.

Listing 2.2 `sales.csv`

```
date,region,product,units,revenue
2024-01-15,North,Widget,100,2999.00
2024-01-15,South,Widget,75,2249.25
2024-01-16,North,Gadget,50,4999.50
```

The first row typically contains column headers. Each subsequent row is a record, with values separated by commas. Simple, portable, universal.

CSV's simplicity is also its limitation. There's no standard way to indicate data types (is "100" a number or text?), handle values containing commas, or represent nested structures. Various dialects exist with different quoting rules and delimiters, which can cause compatibility issues.

2.2.2 JSON: Structured Data as Text

JSON (JavaScript Object Notation) represents structured data with nested objects and arrays. It's become the standard format for web APIs and configuration files.

Listing 2.3 config.json

```
{
  "project": "sales-analysis",
  "database": {
    "host": "localhost",
    "port": 5432,
    "name": "sales_db"
  },
  "output_formats": ["csv", "parquet"],
  "debug": false
}
```

JSON can represent hierarchical data that would be awkward in CSV. It explicitly distinguishes between strings ("text"), numbers (42), booleans (true/false), arrays ([]), and objects ({}). This type information makes parsing more reliable.

2.2.3 YAML and TOML: Configuration Files

YAML and TOML are both used for configuration files, settings that control how programs behave. They prioritize human readability over machine efficiency.

Listing 2.4 config.yaml

```
project: sales-analysis
database:
  host: localhost
  port: 5432
  name: sales_db
output_formats:
  - csv
  - parquet
debug: false
```

Listing 2.5 config.toml

```
[project]
name = "sales-analysis"

[database]
host = "localhost"
port = 5432
name = "sales_db"

[output]
formats = ["csv", "parquet"]

[settings]
debug = false
```

You'll encounter YAML in Quarto document headers and many DevOps tools. TOML appears in Python project configuration (`pyproject.toml`) and Rust's Cargo system. Both are easy to read and edit by hand.

2.2.4 Markdown: Documentation as Text

Markdown (`.md`) is a lightweight way to add formatting to plain text. The text remains readable without any special viewer, but tools can render it with headers, bold text, lists, and links.

Listing 2.6 README.md

```
# Project Title

This project analyzes sales data to identify trends.

## Getting Started

1. Install dependencies
2. Configure the database connection
3. Run the analysis script

See [documentation](docs/guide.md) for details.
```

This book is written in Markdown (specifically, Quarto’s extended Markdown). When you write documentation for your projects, and you will, Markdown is the standard choice.

2.2.5 Excel: The Enterprise Reality

Microsoft Excel files (`.xlsx`) are ubiquitous in business environments. Despite being binary, they’re worth understanding because you’ll inevitably receive data in this format.

An `.xlsx` file is actually a ZIP archive containing XML files that describe the spreadsheet’s content, formatting, and formulas. This complexity means Excel files:

- Support multiple worksheets in one file
- Can contain formatting, formulas, charts, and images
- Are harder to version control or process programmatically
- Sometimes contain hidden surprises (embedded objects, macros)

When possible, prefer exporting data from Excel to CSV for analysis. When you must work with Excel files directly, Python libraries like `openpyxl` can help.

2.2.6 Parquet: The Analytics Format

Parquet is a columnar binary format designed for analytical workloads. Unlike CSV (where data is organized by rows), Parquet stores data by columns, which allows for efficient compression and fast queries that only need certain columns.

You can’t open Parquet files in a text editor, but tools like DuckDB and Python’s `pandas/polars` can read them efficiently. For large datasets, Parquet offers dramatic performance improvements over CSV.

2.3 Directories and File Organization

Files don’t exist in isolation, they live in **directories** (also called folders). Directories can contain files and other directories, creating a hierarchical tree structure.

This tree structure provides organization and namespace separation. You might have multiple files named `data.csv` on your computer, the directory path distinguishes them.

2.3.1 The Root Directory

Every file system has a **root**, the top of the tree from which everything else descends. On macOS and Linux, the root is simply `/`. On Windows, each drive has its own root: `C:\`, `D:\`, etc.

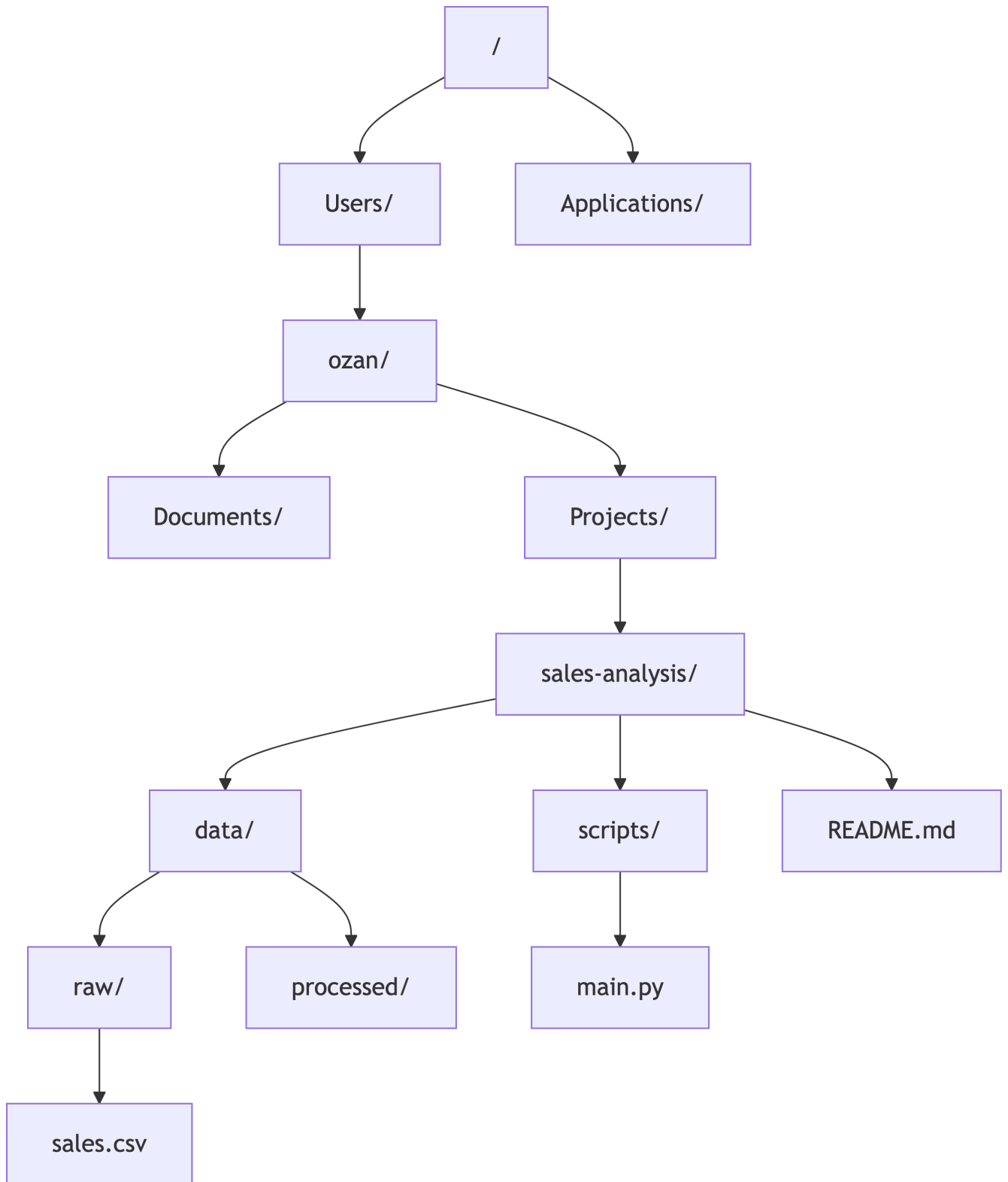


Figure 2.2: Files are organized in a hierarchical tree of directories

2.3.2 Home Directory

Each user on a computer has a **home directory**, a personal space for their files. On macOS, this is typically `/Users/yourname`. On Windows, it's `C:\Users\yourname`. The tilde (`~`) is a universal shortcut meaning “my home directory.”

Your home directory typically contains standard subdirectories like Desktop, Documents, and Downloads, plus hidden configuration files that store preferences for various applications.

2.3.3 Visible and Hidden Files

Files and directories come in two visibility states: **visible** and **hidden**.

Visible items appear normally in file browsers and directory listings. These are the files you work with day-to-day: documents, code, data.

Hidden items have names beginning with a dot (`.`) on macOS/Linux, or a special “hidden” attribute on Windows. They’re excluded from normal directory listings to reduce clutter. Hidden files typically store configuration and preferences that you don’t need to see during normal work.

Examples of hidden files you’ll encounter:

- `.gitignore`, tells Git which files to ignore
- `.git/`, tracks your changes with Git
- `.zshrc`, Zsh shell configuration
- `.venv/`, Python virtual environment directory
- `.DS_Store`, macOS folder metadata (often a nuisance)

To see hidden files:

- **macOS Finder:** Press `Cmd+Shift+.` to toggle visibility
- **Windows Explorer:** View → Show → Hidden items
- **Command line:** We’ll cover this in Chapter 3

Don’t Modify Hidden Files Carelessly

Hidden files are hidden for a reason, they’re typically configuration files that programs depend on. Deleting or modifying them without understanding their purpose can break applications or system features. As you learn more, you’ll become comfortable editing certain hidden files (like `.gitignore`), but approach unknown ones with caution.

2.4 Understanding Paths

A **path** is a string that specifies a location in the file system. Paths are how you tell programs exactly where to find or put files.

2.4.1 Absolute Paths

An **absolute path** specifies a location starting from the root of the file system. It's the complete "address" that works regardless of where you currently are.

macOS/Linux absolute paths start with `/`:

```
/Users/ozan/Projects/sales-analysis/data/sales.csv
```

Windows absolute paths start with a drive letter:

```
C:\Users\ozan\Projects\sales-analysis\data\sales.csv
```

Reading a path from left to right traces a route through the directory tree: start at root, enter Users, enter ozan, enter Projects, and so on until you reach the file.

2.4.2 Relative Paths

A **relative path** specifies a location relative to some starting point, usually your current directory. They're shorter and often more portable.

```
data/sales.csv
```

This means "from wherever I am, go into the data directory, and the file is sales.csv." If you're in `/Users/ozan/Projects/sales-analysis`, this relative path points to `/Users/ozan/Projects/sales-analysis/data/sales.csv`.

2.4.3 Path Separators

The character separating directory names differs by operating system:

- macOS/Linux use forward slash: `/Users/ozan/file.txt`
- Windows uses backslash: `C:\Users\ozan\file.txt`

This difference causes occasional headaches when sharing code between systems. Python's `pathlib` module handles this automatically, which is why we'll use it when writing code that works with files.

2.4.4 Special Path Symbols

Several symbols have special meaning in paths:

Table 2.1: Special path symbols and their meanings

Symbol	Meaning	Example
.	Current directory	<code>./script.py</code>
..	Parent directory	<code>../other-project/</code>
~	Home directory	<code>~/Documents/</code>

The double-dot (`..`) is particularly useful for navigating “up” the tree:

```
../../shared/data/reference.csv
```

This means: go up one directory, go up another directory, then descend into `shared`, then `data`, and find `reference.csv`.

2.4.5 Path Best Practices

When organizing your own projects:

1. **Avoid spaces** in file and directory names. Use hyphens (`my-project`) or underscores (`my_project`) instead. Spaces require special handling in many tools.
2. **Use lowercase** for directories and files when possible. Some systems are case-sensitive (`Data` and `data` are different), others aren’t. Lowercase avoids confusion.
3. **Be consistent** with naming conventions within a project. Pick a style and stick with it.
4. **Keep paths reasonable** in length. Deeply nested structures with long names become unwieldy.

2.5 Project Organization

How you organize files within a project affects how easily you (and others) can understand and work with it. A well-organized project is easier to navigate, debug, and share.

A typical data project structure might look like:

```
sales-analysis/
  README.md          # Project description and instructions
  data/
    raw/             # Original, unmodified data
    processed/       # Cleaned/transformed data
  scripts/           # Python/SQL code
  outputs/           # Generated reports, figures
  docs/              # Additional documentation
```

Key principles:

- **Separate raw from processed data.** Never modify original data files; create processed versions instead. This ensures you can always reproduce your work from the source.
- **Keep code in one place.** Don't scatter scripts throughout the project.
- **Include a README.** Anyone opening your project should immediately understand what it is and how to use it.
- **Put outputs in their own directory.** Generated files (reports, figures, exports) should be clearly distinguished from source materials.

We'll revisit project organization throughout this course, especially when we learn Git for version control in [Chapter 4](#).

2.6 Summary

Files are the fundamental unit of persistent storage on computers, named collections of binary data that programs read and write. Understanding files as either text (human-readable) or binary (requiring specialized interpretation) helps you choose appropriate tools and formats for different tasks.

The file system organizes files into a hierarchical tree of directories. Every location can be specified by a path, either absolute (from the root) or relative (from some starting point). Special symbols like `.`, `..`, and `~` provide shortcuts for common navigation patterns.

Different file formats serve different purposes: CSV for universal tabular data exchange, JSON for structured data and APIs, YAML and TOML for configuration, Markdown for documentation, and specialized formats like Parquet for high-performance analytics. Knowing what's inside each format, and whether you can open it in a text editor, guides how you work with it.

With this foundation, you're ready to explore the command line in [Chapter 3](#), where you'll navigate this file system and manipulate files by typing commands rather than clicking through graphical interfaces.

2.7 Glossary

Absolute path A path that specifies a location starting from the root of the file system, such as `/Users/ozan/Documents/file.txt`. Works regardless of current directory.

Binary file A file whose contents are not meant to be interpreted as text characters. Examples include images, compiled programs, and Excel spreadsheets.

CSV (Comma-Separated Values) A text-based format for tabular data where values are separated by commas and rows by newlines.

Directory A container in the file system that can hold files and other directories. Also called a folder.

File A named collection of data stored on a disk or other storage medium.

File extension The suffix after the dot in a filename (like `.csv` or `.py`) that indicates the file's format.

File system The method and structure an operating system uses to organize and store files on storage devices.

Hidden file A file (typically starting with `.`) that doesn't appear in normal directory listings. Often used for configuration.

Home directory A user's personal directory, represented by `~`. On macOS: `/Users/username`. On Windows: `C:\Users\username`.

JSON (JavaScript Object Notation) A text-based format for structured data that supports nested objects, arrays, and typed values.

Markdown A lightweight text formatting syntax that remains human-readable while allowing rendering into formatted documents.

Path A string that specifies a location in the file system by listing the directories (and optionally filename) to traverse.

Path separator The character used to separate directory names in a path. Forward slash (`/`) on macOS/Linux, backslash (`\`) on Windows.

Relative path A path specified relative to some starting location (usually the current directory) rather than from the file system root.

Root directory The top-level directory of a file system, from which all other directories descend. Represented as `/` on macOS/Linux.

Text file A file whose contents are encoded as human-readable characters. Can be opened and read in any text editor.

YAML/TOML Text-based configuration file formats designed for human readability.

2.8 Exercises

2.8.1 Question 2.1

Which of the following is a **text file** that can be opened and read in any text editor?

- A) .xlsx (Excel spreadsheet)
- B) .jpg (image file)
- C) .csv (comma-separated values)
- D) .pdf (portable document format)

2.8.2 Question 2.2

What does an **absolute path** specify?

- A) A location relative to the current directory
- B) A location starting from the root of the file system
- C) A location relative to the home directory
- D) A location that changes based on the operating system

2.8.3 Question 2.3

On macOS and Linux, what character is used to separate directories in a path?

- A) Backslash \
- B) Forward slash /
- C) Colon :
- D) Period .

2.8.4 Question 2.4

What does the symbol ~ represent in a file path?

- A) The root directory
- B) The current directory

- C) The home directory
- D) The parent directory

2.8.5 Question 2.5

What does the symbol `..` represent in a file path?

- A) The root directory
- B) The current directory
- C) The home directory
- D) The parent directory (one level up)

2.8.6 Question 2.6

Why are hidden files (those starting with `.`) hidden by default?

- A) They contain viruses and malware
- B) They are typically configuration files that reduce clutter when hidden
- C) They are encrypted and cannot be read
- D) They are system files that will crash the computer if viewed

2.8.7 Question 2.7

What is TRUE about file extensions like `.csv` or `.py`?

- A) They determine what programs can create the file
- B) They are enforced by the operating system and cannot be changed
- C) They are conventions that tell programs how to interpret the file
- D) They affect the actual binary content of the file

2.8.8 Question 2.8

Which file format is described as “actually a ZIP archive containing XML files”?

- A) .csv
- B) .json
- C) .xlsx
- D) .parquet

2.8.9 Question 2.9

Why does version control (like Git) work better with text files than binary files?

- A) Binary files are too large to store
- B) Text files can track line-by-line changes meaningfully
- C) Binary files cannot be committed to repositories
- D) Text files compress better than binary files

2.8.10 Question 2.10

Given the path `../../shared/data/file.csv`, how many directory levels up does this path navigate before descending?

- A) 0
- B) 1
- C) 2
- D) 3

3 The Command Line

Every engineer has two ways to communicate with their computer: pointing and clicking through windows and icons, or typing commands directly. The first approach, the graphical user interface (GUI), feels natural because it mirrors the physical world. You see folders, drag files, click buttons. But this intuitive interface hides something powerful underneath: a text-based system where you can tell your computer exactly what to do, one command at a time.

This text-based interface is called the **command line**, and learning it transforms you from a passenger into a driver of your computational environment. Instead of being limited to the options someone else designed into a menu, you gain direct access to your computer's capabilities. You can automate repetitive tasks, chain tools together in ways their creators never imagined, and work with the same professional tooling used by software engineers, data scientists, and researchers worldwide.

The command line might feel foreign at first, you're used to clicking, not typing. But consider this: every supercomputer in the world runs Linux and is operated through a command line. Every cloud server you'll ever deploy to. Every professional data pipeline, every version control system, every modern development tool. The command line isn't just a historical artifact; it's the universal interface that connects everything in computational work.

By the end of this chapter, you'll understand how to navigate your filesystem, run programs, and use the foundational commands that make tools like Git, Quarto, and Python package managers possible. More importantly, you'll understand *why* the command line works the way it does, knowledge that transfers to any tool you encounter in the future.

3.1 Why Learn the Command Line?

Before diving into commands, let's understand what makes this skill worth learning. The command line offers four distinct advantages that matter for building data products.

The command line enables agility. Data work is exploratory. You try something, see what happens, adjust, and try again. The command line supports this through what programmers call a “read-eval-print loop” (REPL): you type a command, press Enter, and immediately see the result. This rapid feedback loop, much faster than clicking through menus, matches the experimental nature of working with data. When you need to quickly check a file's contents,

count rows in a dataset, or test whether a transformation worked, the command line lets you do it in seconds.

The command line connects your tools. You'll learn Python, SQL, Quarto, and Git in this book. Each is powerful alone, but the real magic happens when they work together. The command line acts as the glue: you can take output from one tool and feed it to another, orchestrate complex workflows, and build pipelines that combine multiple technologies. A Python script can call a shell command; a shell script can run Python code. This interoperability is only possible because everything speaks the common language of the command line.

The command line scales and repeats. Clicking through a GUI works fine once. But what if you need to process 500 files the same way? What if a colleague needs to reproduce your analysis? What if you need to run the same task every night? Everything you type on the command line can be saved to a script and run automatically, on any schedule, on any machine with the same tools installed. This transforms manual, one-time tasks into automated, reproducible workflows, exactly what professional data products require.

The command line is everywhere. The skills you learn here work on your laptop, on a cloud server, on a Raspberry Pi, on a supercomputer. Linux and Unix-style commands have been the standard for over 50 years and remain the foundation of modern computing infrastructure.

3.2 The Layers of Your Computing Environment

When you open a terminal application and start typing commands, several layers of software work together to make things happen. Understanding these layers helps demystify what's actually occurring when you press Enter.

The terminal (or terminal emulator) is the application window where you type. It's the visual interface, handling the display of text, accepting keyboard input, and showing you output. On macOS, the default is called [Terminal](#). On Windows, we'll use [Windows Terminal](#). The terminal is just the window; the real work happens in the layers below.

The shell is the program that interprets what you type. When you enter a command, the shell parses your input, figures out what program you want to run, and handles things like connecting programs together or managing variables. Different shells exist, Bash, Zsh, PowerShell, Fish, each with slightly different features. Think of the shell as the interpreter between your human-readable commands and the system calls that actually run programs.

Command-line tools are the programs the shell invokes on your behalf: `ls` to list files, `cd` to change directories, `python` to run Python code, `git` to manage version control. These are the workhorses, each tool does something specific, and you'll build up a personal toolkit of favorites.

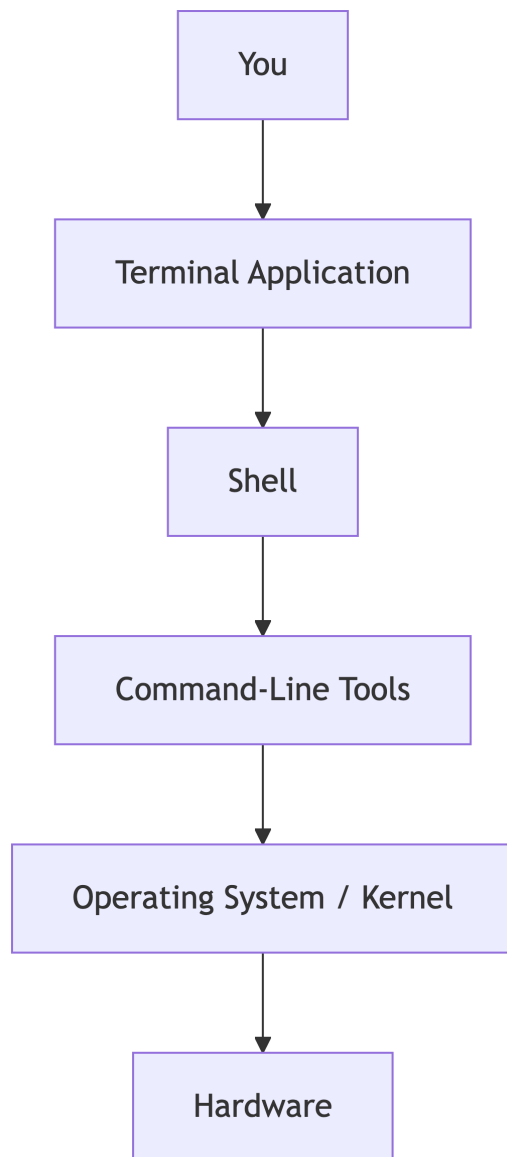


Figure 3.1: The layers between you and your computer's hardware, from user interaction down to hardware

The operating system sits beneath everything, managing hardware resources and executing programs. The kernel, the core of the operating system, handles the actual work of running commands, reading files from disk, and managing memory. When a command-line tool needs to read a file, it asks the kernel; the kernel talks to the hardware.

You don't need to think about these layers constantly, but understanding them explains why certain things work the way they do. When a command "isn't found," the problem is usually in how the shell locates programs. When a terminal looks different from someone else's, it's often just different terminal settings, the underlying commands still work the same way.

3.3 Setting Up Your Terminal

The specific application you use to access the command line depends on your operating system, but the commands you'll learn work across both platforms. This cross-platform compatibility exists because the major shells have converged on a common set of basic commands, what we might call the "Unix vocabulary."

3.3.1 macOS Users

macOS comes with a terminal application called Terminal, located in Applications → Utilities. Open it and you'll see a window with a prompt waiting for input. Modern macOS uses Zsh (Z shell) as its default shell, which provides an excellent experience with helpful features like better tab completion and spelling correction.

Your default prompt probably shows your username and computer name, followed by a % symbol. This prompt tells you the shell is ready to accept a command.

3.3.2 Windows Users

On Windows, we'll use **Windows Terminal** with **PowerShell** as the shell. Windows Terminal is Microsoft's modern terminal application that provides a clean, tabbed interface for command-line work. PowerShell is the shell that runs inside it.

To get started, install [Windows Terminal from the Microsoft Store](#) if it's not already on your system. When you open Windows Terminal, it launches PowerShell by default, you'll see a prompt showing your current location followed by a > symbol.

! Use Windows Terminal, Not the Legacy Apps

Windows has several command-line applications, which can be confusing. **Always use Windows Terminal** for this book. Avoid the standalone "PowerShell" application (blue

icon), “Command Prompt” (black icon), and “Git Bash.” These older applications work but lack Windows Terminal’s modern features. Windows Terminal gives you a consistent experience and will be what you use inside code editors like VS Code or Zed as well.

The key insight for Windows users: PowerShell includes built-in aliases that map Unix command names to their PowerShell equivalents. When you type `ls`, PowerShell translates this to its native `Get-ChildItem` command. When you type `pwd`, it runs `Get-Location`. This means the basic commands in this book work on both platforms, even though the underlying implementations differ.

3.3.3 The Prompt

Regardless of your operating system, you’ll see a **prompt**, a short piece of text indicating the shell is ready for input. The prompt typically shows useful information like your current directory location, but its exact appearance varies based on your system and configuration.

Throughout this book, I’ll show commands preceded by a `$` symbol:

Listing 3.1 terminal

```
$ whoami
```

This dollar sign represents the prompt, it’s not something you type. It’s simply a convention to indicate “this is a command you should enter.” When you see `$ whoami`, you type only `whoami` and press Enter.

Some examples will show both the command and its output:

```
$ whoami
ozan

$ date
Thu Jan 29 14:30:00 EST 2026
```

The lines with `$` are commands; the lines without are the output those commands produced.

3.4 Navigating the Filesystem

At any moment in your terminal session, you exist in a particular location within your computer’s filesystem, a specific directory. This is your **current working directory**, and it affects how

many commands behave. When you list files, you see the files in your current directory. When you create a file without specifying a full path, it appears in your current directory.

3.4.1 Where Am I? The `pwd` Command

The `pwd` command (print working directory) tells you exactly where you are:

3.5 macOS/Linux

Listing 3.2 terminal

```
$ pwd
/Users/ozan
```

3.6 Windows

Listing 3.3 terminal

```
> pwd

Path
----
C:\Users\ozan
```

This output is a **path**, a string that describes a location in your filesystem. The slashes separate directory names. Reading from left to right traces a route through the directory tree: start at the root, go into the Users directory, then into the ozan directory.

When you first open a terminal, you typically start in your **home directory**, your personal space on the computer. On macOS, this is usually `/Users/yourname`. On Windows, it's usually `C:\Users\yourname`. The tilde character (`~`) is a universal shortcut that means “my home directory.”

3.6.1 What's Here? The `ls` Command

The `ls` command (list) shows the contents of your current directory:

Listing 3.4 terminal

```
$ ls
Desktop      Downloads    Music        Public
Documents    Library      Pictures
```

By default, `ls` shows only the names of visible files and directories. The output format differs slightly between macOS/Linux and Windows, but the information is similar.

3.7 macOS/Linux

To see more detail, add the `-l` flag (for “long” format):

Listing 3.5 terminal

```
$ ls -l
total 0
drwx-----@  4 ozan  staff   128 Jan 20 10:15 Desktop
drwx-----@ 12 ozan  staff  384 Jan 23 14:22 Documents
drwx-----+  5 ozan  staff  160 Jan 21 09:00 Downloads
```

To see hidden files (those starting with `.`), add the `-a` flag (for “all”):

Listing 3.6 terminal

```
$ ls -a
.          .zshrc      Downloads   Pictures
..         Desktop   Library     Public
.config    Documents   Music
```

You can combine flags: `ls -la` shows all files including hidden ones in long format.

3.8 Windows

PowerShell’s default `ls` output already shows detailed information including Mode, LastWrite-Time, Length, and Name, similar to the `-l` flag on macOS/Linux.

To see hidden files, add the `-Force` flag:

Listing 3.7 terminal

```
> ls -Force
```

```
Directory: C:\Users\ozan
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
d--h--	1/20/2026 10:15 AM		.config
d-----	1/23/2026 2:22 PM		Desktop
d-----	1/21/2026 9:00 AM		Documents
d-----	1/21/2026 9:00 AM		Downloads

Files with **h** in the Mode column are hidden.

i Flags and Options

Command-line tools accept **flags** (also called options or switches) that modify their behavior. Flags typically start with a dash. Short flags use a single dash and single letter (**-l**, **-a**). Long flags use two dashes and a word (**--all**, **--help**). The specific flags available vary by command and sometimes by operating system.

3.8.1 Moving Around: The **cd** Command

The **cd** command (change directory) moves you to a different location:

Listing 3.8 terminal

```
$ cd Documents
$ pwd
/Users/ozan/Documents
```

After running **cd Documents**, your current directory changes. Now commands like **ls** show the contents of the Documents folder.

You can navigate using either **relative paths** (relative to your current location) or **absolute paths** (starting from the root of the filesystem). The command **cd Documents** uses a relative path, it means “go into the Documents folder that’s inside my current directory.” The command **cd /Users/ozan/Documents** (or **cd C:\Users\ozan\Documents** on Windows) uses an absolute path, it works regardless of where you currently are.

Several special shortcuts make navigation easier:

Table 3.1: Navigation shortcuts in the command line

Symbol	Meaning
.	Current directory
..	Parent directory (one level up)
~	Home directory
-	Previous directory (macOS/Linux only)

Let's see these in action:

Listing 3.9 terminal

```
$ pwd
/Users/ozan/Documents

$ cd ..
$ pwd
/Users/ozan

$ cd ~
$ pwd
/Users/ozan

$ cd ~/Downloads
$ pwd
/Users/ozan/Downloads
```

The double-dot (..) moved up one level, from Documents to the home directory. The tilde took us home (which was the same place in this case). Then ~/Downloads navigated directly to Downloads using an absolute path starting from home.

3.8.2 Tab Completion: Your Best Friend

Here's a technique that will save you thousands of keystrokes: **tab completion**. Instead of typing full directory or file names, type the first few characters and press Tab. The shell will complete the name if it's unambiguous, or show you the options if multiple matches exist.

If you type `cd D` and press Tab, the shell might show you Desktop, Documents, and Downloads. Type one more character to disambiguate (`cd Do<Tab>`) and Tab again.

Listing 3.10 terminal

```
$ cd Doc<Tab>
$ cd Documents/
```

Tab completion also works for command names and flags. Make it a habit, professionals use it constantly.

3.9 Working with Files and Directories

Now that you can navigate, let's manipulate the filesystem: creating, moving, and removing files and directories.

3.9.1 Creating Directories: `mkdir`

The `mkdir` command (make directory) creates a new folder:

Listing 3.11 terminal

```
$ mkdir my-project
$ ls
Desktop      Downloads    Music        Public        my-project
Documents    Library      Pictures
```

A new directory called `my-project` now exists in your current location.

Naming Conventions

Avoid spaces in file and directory names. While spaces are technically allowed, they complicate command-line work because you need special handling (quotes or backslashes) to reference them. Use hyphens (`my-project`), underscores (`my_project`), or camelCase (`myProject`) instead.

3.9.2 Removing Files and Directories: `rm` and `rmdir`

The `rm` command removes files:

Listing 3.12 terminal

```
$ rm my-project/temp.txt
```

 There Is No Recycle Bin

Unlike deleting through the GUI, `rm` does not move files to a trash folder. Removal is immediate and permanent. There’s no “undo.” Double-check your commands, especially when using wildcards (patterns like `*.txt` that match multiple files).

To remove directories, use `rmdir` for empty directories or `rm -r` for directories with contents:

Listing 3.13 terminal

```
$ rmdir empty-folder          # Only works if folder is empty
$ rm -r folder-with-contents  # Removes everything inside recursively
```

The `-r` flag means “recursive”, delete this directory and everything inside it.

3.10 Understanding Paths

Paths are how you specify locations in the filesystem. Mastering them removes much of the confusion beginners experience. If you’ve read Chapter 2, you’ll recognize these concepts.

An **absolute path** starts from the root of the filesystem and provides the complete route to a location:

- macOS/Linux: `/Users/ozan/Documents/project/data.csv`
- Windows: `C:\Users\ozan\Documents\project\data.csv`

Absolute paths work from anywhere, they don’t depend on your current directory.

A **relative path** starts from your current directory:

- `Documents/project/data.csv` (if you’re in `/Users/ozan`)
- `../project/data.csv` (if you’re in `/Users/ozan/other-folder`)
- `./script.py` (file in current directory; the `./` is optional but explicit)

The dot (`.`) represents your current directory. The double-dot (`..`) represents the parent directory. You can chain these: `../../other` means “go up two levels, then into other.”

When specifying paths with spaces (which you should avoid creating, but sometimes encounter), wrap the path in quotes:

Listing 3.14 terminal

```
$ cd "My Documents"
$ ls "file with spaces.txt"
```

3.11 Commands, Arguments, and Options

Every command you type follows a pattern: the command name, possibly some options (flags), and possibly some arguments (values the command operates on).

```
command [options] [arguments]
```

Consider this command:

Listing 3.15 terminal

```
$ ls -la ~/Documents
```

- `ls` is the command
- `-la` combines two options: `-l` (long format) and `-a` (all files)
- `~/Documents` is the argument (the directory to list)

Options modify behavior; arguments specify targets. Some commands require arguments (`cd` needs to know where to go). Some commands have sensible defaults when arguments are omitted (`ls` without arguments lists the current directory).

Most commands support a `--help` flag that displays usage information:

Listing 3.16 terminal

```
$ ls --help
$ git --help
$ python --help
```

3.12 Types of Commands

Not everything you type in the terminal is the same kind of thing. Understanding the types of command-line tools helps demystify what's happening when you run commands.

Binary executables are compiled programs, the classic notion of software. When you run `python` or `git`, you're invoking a binary executable that was compiled from source code. These live somewhere on your system and the shell knows how to find them.

Shell builtins are commands built into the shell itself. Commands like `cd` and `pwd` are often builtins rather than separate programs. They're part of the shell, so they can modify the shell's internal state (like the current directory).

Interpreted scripts are text files that get executed by an interpreter. A Python script is run by the Python interpreter; a Bash script by the Bash shell. When you write a script, you're creating a new command-line tool.

Aliases are shortcuts, one command that expands into another. PowerShell's ability to let you type `ls` and have it run `Get-ChildItem` is achieved through aliases.

You can often determine what type something is using the `type` command:

Listing 3.17 terminal

```
$ type cd
cd is a shell builtin

$ type python
python is /usr/local/bin/python
```

3.13 Getting Help

When you encounter an unfamiliar command or forget the options for a familiar one, several strategies help.

The `--help` flag works with most commands and provides a brief usage summary:

Listing 3.18 terminal

```
$ git --help
$ python --help
```

Web searches for “how to [task] command line” or “[command name] examples” usually surface useful Stack Overflow answers and tutorials.

As you gain experience, you’ll develop intuition for standard conventions: `-v` often means verbose, `-q` means quiet, `-f` specifies a file, `-o` specifies output. But every tool is different, so checking documentation beats guessing.

3.14 A Taste of What’s Coming

The commands covered in this chapter, `pwd`, `ls`, `cd`, `mkdir`, `rm`, form your foundational vocabulary. With them, you can navigate the filesystem and manage files. But the real power of the command line emerges when you combine these basics with more sophisticated tools.

In the coming chapters, you’ll use this foundation to:

- Initialize Git repositories and track changes to your projects (see Chapter 4)
- Run Quarto to transform Markdown files into beautiful documents and websites
- Use `uv` to create Python virtual environments and manage dependencies
- Execute Python scripts and SQL queries from the command line
- Build automated pipelines that orchestrate multiple tools

Each of these builds on the patterns you’ve learned here: typing commands, passing options and arguments, navigating to the right directory, and understanding how your shell finds and runs programs.

The command line rewards regular practice. At first, you’ll type slowly and check documentation constantly. Within weeks, muscle memory takes over. Commands that initially seemed cryptic become natural vocabulary. The efficiency gains compound: what takes ten clicks in a GUI takes one command on the command line, and that command can run automatically while you sleep.

3.15 Summary

This chapter introduced the command line as the text-based interface for interacting with your computer. You learned why this skill matters for building data products, namely agility, tool integration, scalability, and ubiquity, and explored the layered architecture from terminal to shell to operating system.

The core navigation commands (`pwd`, `ls`, `cd`) let you understand and traverse the filesystem. File manipulation commands (`mkdir`, `rm`) let you create and delete files and directories. Understanding paths (absolute vs. relative) and special symbols (`.`, `..`, `~`) enables you to reference locations anywhere in the filesystem.

Commands follow a consistent pattern of `command [options] [arguments]`, and options/flags modify behavior while arguments specify targets.

With this foundation, you're ready to learn Git (version control) in Chapter 4, Quarto (document rendering), and the other command-line tools that form the backbone of modern data workflows.

3.16 Glossary

Absolute path A path that starts from the root of the filesystem and provides the complete route to a location, such as `/Users/ozan/Documents/project`. Absolute paths work regardless of your current directory.

Alias A shortcut that makes one command expand into another. PowerShell uses aliases to let Unix-style commands like `ls` work on Windows.

Argument A value passed to a command that specifies what the command should operate on. In `ls Documents`, the argument is `Documents`.

Binary executable A compiled program stored as machine code that can be run directly by the operating system.

Command line interface (CLI) A text-based interface for interacting with a computer by typing commands rather than using graphical elements.

Current working directory The directory location you're currently in within the terminal. Commands operate relative to this location unless you specify absolute paths.

Flag An option passed to a command that modifies its behavior, typically starting with a dash. Also called an option or switch.

Graphical user interface (GUI) The visual interface of windows, icons, menus, and buttons that lets you interact with a computer by pointing and clicking.

Hidden file A file whose name starts with a dot (`.`) on macOS/Linux, or has the hidden attribute on Windows, making it invisible to normal directory listings.

Home directory Your personal directory on the computer, represented by the `~` symbol. On macOS: `/Users/yourname`. On Windows: `C:\Users\yourname`.

Kernel The core of an operating system that manages hardware resources and executes programs.

Prompt The text displayed by the shell to indicate it's ready for input. Often shows information like the current directory.

Read-eval-print loop (REPL) An interactive environment that reads input, evaluates it, prints the result, and loops back for more input.

Relative path A path specified relative to the current directory rather than from the filesystem root. Examples include `Documents/file.txt` or `../other-folder`.

Root directory The top-level directory of a filesystem, represented by `/` on macOS/Linux or a drive letter like `C:\` on Windows.

Shell A program that interprets commands you type and executes them. Examples include Zsh, Bash, and PowerShell.

Shell builtin A command that's built into the shell itself rather than being a separate program. `cd` is typically a builtin.

Tab completion A feature where pressing Tab auto-completes file names, directory names, and commands based on what you've typed so far.

Terminal The application that provides a window for entering commands and viewing output. It communicates with the shell.

Windows Terminal Microsoft's modern terminal application for Windows that provides a clean interface for command-line shells like PowerShell.

3.17 Exercises

3.17.1 Question 3.1

What is the relationship between a **terminal** and a **shell**?

- A) They are the same thing
- B) The terminal is the window; the shell interprets commands
- C) The shell is the window; the terminal interprets commands
- D) The terminal runs inside the shell

3.17.2 Question 3.2

What does the `pwd` command do?

- A) Print the contents of a file
- B) Print the current working directory
- C) Create a new directory
- D) Delete a file

3.17.3 Question 3.3

What does the `ls` command do?

- A) List the contents of a directory

- B) Move to a different directory
- C) Create a new file
- D) Remove a directory

3.17.4 Question 3.4

What does the `cd` command do?

- A) Copy a directory
- B) Create a directory
- C) Change the current directory
- D) Clear the terminal display

3.17.5 Question 3.5

In the command `ls -la ~/Documents`, what is `-la`?

- A) An argument specifying a directory
- B) A path to a file
- C) Options/flags that modify the command's behavior
- D) The name of a file to list

3.17.6 Question 3.6

What happens when you delete a file using `rm` on the command line?

- A) The file moves to the Recycle Bin/Trash
- B) The file is permanently deleted with no undo
- C) The file is backed up before deletion
- D) The file is marked as hidden

3.17.7 Question 3.7

What is **tab completion**?

- A) A feature that creates new tabs in the terminal
- B) A feature that auto-completes file and command names when you press Tab
- C) A feature that indents code automatically
- D) A feature that opens multiple terminals

3.17.8 Question 3.8

Which of the following is an advantage of using the command line over a GUI?

- A) Command line interfaces are more visually appealing
- B) Commands can be saved to scripts and automated
- C) Command line interfaces require less learning
- D) Command line interfaces work without a keyboard

3.17.9 Question 3.9

What is a **shell builtin**?

- A) A program installed by the package manager
- B) A command built into the shell itself, like `cd`
- C) A hardware component of the computer
- D) A graphical element of the terminal window

3.17.10 Question 3.10

The command `cd ..` will:

- A) Move to the home directory

- B) Move to the root directory
- C) Move up one directory level (to the parent)
- D) Stay in the current directory

4 Version Control with Git

Every engineer eventually faces a moment of dread: you made changes to a working project, and now nothing works. You desperately wish you could go back to the version from yesterday, or last week, or before that “quick fix” that broke everything. This is the problem that version control solves.

Git is a *version control system* that tracks changes to files over time. Think of it as an infinitely powerful “undo” system combined with a detailed logbook of every modification ever made to your project. But Git goes far beyond simple backups. It enables you to experiment fearlessly, collaborate with teammates without overwriting each other’s work, and maintain a complete, searchable history of how your project evolved.

Git was created in 2005 by Linus Torvalds to manage the Linux kernel, one of the largest collaborative software projects in history. The tool was designed for speed, data integrity, and supporting distributed, non-linear workflows. These design principles make Git equally valuable for a solo engineer tracking changes to a simulation model or a team collaborating on a production system.

4.1 Why Not Just Use Google Docs?

You’re probably familiar with Google Docs and its automatic saving feature. Every keystroke is saved to the cloud, and you can see a version history showing how the document evolved. This works wonderfully for prose documents like essays, reports, and meeting notes. So why do we need Git?

The answer lies in the nature of what we’re tracking. Google Docs excels at collaborative prose writing where changes flow continuously and merge naturally. If two people edit different paragraphs simultaneously, Google Docs handles it seamlessly. The autosave feature means you never lose work, and the version history lets you see how the document evolved.

But consider what happens with code and technical projects. Code is not prose. A single misplaced character can break an entire program. Changes to one file often depend on changes to other files. You need to test changes before committing to them. And you frequently need to work on experimental features without risking your stable, working version.

Here’s a comparison of the two approaches:

Aspect	Google Docs	Git
Saving	Automatic, continuous	Manual, intentional
History	Every keystroke recorded	Meaningful snapshots you define
Branching	Not supported	Core feature
Offline work	Limited	Full functionality
File types	Documents only	Any file type
Collaboration	Real-time, same document	Asynchronous, merge when ready

Google Docs’ autosave is perfect for prose because you want every word captured as you write. But for code, you want *intentional* checkpoints. You want to save a version when the code works, not capture every typo and false start along the way. Git gives you this control: you decide when to create a checkpoint (called a “commit”) and what message describes that checkpoint.

The branching capability is equally important. Imagine you have a working report and want to try a completely different organizational structure. In Google Docs, you’d make a copy of the entire document. With Git, you create a “branch,” an independent line of development. If the experiment works, you merge it back. If it fails, you simply delete the branch. Your original remains untouched either way.

4.2 The Git Mental Model

Before diving into commands, let’s build a clear mental model of how Git organizes your work. Understanding this structure will make every Git operation intuitive rather than mysterious.

4.2.1 The Three States

Every file in a Git project exists in one of three states, which correspond to three areas where files live:

1. **Working Directory:** This is the actual folder on your computer containing your project files. When you edit a file in Zed, you’re modifying the working directory. Git can see these files but hasn’t recorded the changes yet.

2. **Staging Area** (also called the “Index”): This is a preparation zone where you gather changes before recording them. Think of it as a draft of your next commit. You explicitly choose which changes to stage, giving you precise control over what gets recorded.
3. **Repository** (the `.git` directory): This is Git’s database, stored in a hidden `.git` folder in your project. It contains the complete history of your project: every commit, every version of every file, and all the metadata about who changed what and when.

The basic Git workflow moves changes through these three states:

1. You modify files in your **working directory**
2. You *stage* the changes you want to record, adding them to the **staging area**
3. You *commit* the staged changes, which permanently stores a snapshot in the **repository**

⚠ Save Your Files!

Git only sees changes that have been saved to disk. If you edit a file in Zed but don’t save it (Ctrl+S or Cmd+S), Git has no idea the file changed. This is different from Google Docs, where every keystroke is automatically saved. Before running any Git command, make sure you’ve saved your files. Zed shows a dot on the tab of unsaved files, so look for that indicator. For more about the disk vs. memory distinction, see Chapter 1.

4.2.2 Commits: The Building Blocks

A commit is a snapshot of your project at a specific moment. Each commit contains the state of every tracked file, a timestamp, the author’s name and email, a message describing the changes, and a unique identifier (a 40-character hash like `a1b2c3d4...`).

Commits form a chain, with each commit pointing back to its parent (the previous commit). This chain is your project’s history, and you can navigate it to see how your project evolved or to restore previous states.

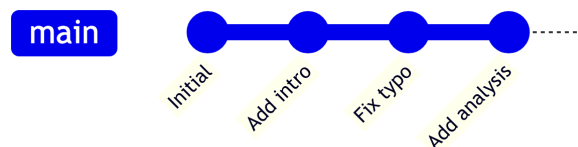


Figure 4.1: Commits form a linked chain representing your project’s history

4.3 Your First Git Project

Let's create a project and track it with Git. We'll build a technical report using Markdown, something you already know how to do. This example demonstrates Git's value even for documents, and the same workflow applies exactly when we move to code later.

4.3.1 Initializing a Repository

First, create a new directory for your project. Open your terminal (see Chapter 3 if you need a refresher) and navigate to where you want the project to live:

Listing 4.1 terminal

```
mkdir quality-report  
cd quality-report
```

Now initialize a Git repository in this directory:

Listing 4.2 terminal

```
git init
```

Git responds with confirmation:

```
Initialized empty Git repository in C:/Users/yourname/quality-report/.git/
```

This command created a hidden `.git` directory containing Git's database. Your project folder looks empty, but Git is now watching it, ready to track changes.

In Zed: You can also open Zed, create a new folder, and then open the terminal panel (Ctrl+‘ or View → Terminal) to run `git init`. Zed will automatically detect that the folder is now a Git repository and enable its Git features.

4.3.2 Checking Repository Status

The `git status` command is your window into Git's perspective. Run it frequently to understand what's happening:

Listing 4.3 terminal

```
git status
```

On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)

Git tells us we're on a branch called `main` (more on branches later), we haven't made any commits yet, and there's nothing to commit because we haven't created any files.

In Zed: The Source Control panel (click the branch icon in the left sidebar, or press `Ctrl+Shift+G`) shows similar information. It displays changed files, staged files, and lets you perform Git operations with clicks instead of commands.

4.3.3 Creating and Tracking Files

Let's create our report. In Zed, create a new file (`File → New File`, or `Ctrl+N`) and immediately save it as `report.md` in your `quality-report` folder (`File → Save`, or `Ctrl+S`). Add some initial content:

Listing 4.4 `report.md`

```
# Manufacturing Quality Analysis Report

## Executive Summary

This report analyzes defect rates across three production lines
during Q4 2024.
```

Save the file (`Ctrl+S`). This is crucial: Git cannot see your changes until they exist on disk.

Now check the status in your terminal:

Listing 4.5 terminal

```
git status
```

On branch main

No commits yet

Untracked files:

(use "git add <file>..." to include in what will be committed)
report.md

nothing added to commit but untracked files present (use "git add" to track)

Git sees the new file but labels it as “untracked.” Git doesn’t automatically track every file in your directory. You must explicitly tell Git which files to track. This is a feature, not a limitation. It lets you keep temporary files, large datasets, or sensitive configuration files out of version control.

In Zed: The Source Control panel shows `report.md` under “Untracked Files”. In the file explorer, the filename may appear in a different color (often green) to indicate it’s new and untracked.

4.3.4 Staging Changes

To start tracking a file and prepare it for your first commit, use `git add`:

Listing 4.6 terminal

```
git add report.md
```

Check the status:

Listing 4.7 terminal

```
git status
```

On branch main

No commits yet

Changes to be committed:

(use "git rm --cached <file>..." to unstage)
new file: report.md

The file moved from “untracked” to “changes to be committed.” It’s now in the staging area, ready to be recorded in a commit.

In Zed: You can stage files by clicking the checkbox icon next to the filename in the Source Control panel, or by right-clicking and selecting “Stage File”. The file moves from the “Changes” section to the “Staged Changes” section.

4.3.5 Making Your First Commit

Create a commit with the `git commit` command. Every commit requires a message describing what changed. Use the `-m` flag to provide the message inline:

Listing 4.8 terminal

```
git commit -m "Start quality analysis report with executive summary"
```

```
[main (root-commit) a1b2c3d] Start quality analysis report with executive summary
1 file changed, 6 insertions(+)
create mode 100644 report.md
```

You’ve made your first commit. Git recorded a snapshot of your project that you can return to at any time.

In Zed: Type your commit message in the text box at the bottom of the Source Control panel, then click “Commit Tracked”. Zed calls the same `git commit` command behind the scenes.

Zed Uses Git, Not the Other Way Around

Here’s an important mental model: Zed is a program that knows how to talk to Git. When you click “Commit Tracked” in Zed, it runs `git commit` for you. But Git itself has no idea Zed exists. Git is a standalone program that works entirely through files on disk and command-line instructions.

This means you can mix and match: use Zed’s GUI for some operations, the terminal for others, or even a different editor entirely. They all work with the same Git repository because they’re all just interfaces to the same underlying Git program.

If you delete Zed tomorrow, Git still works perfectly from the command line. If Git breaks, Zed’s Source Control panel won’t be able to help you because it depends on Git being available. Understanding this relationship will help you troubleshoot problems: if something isn’t working in Zed’s Git panel, try the command in the terminal to see the actual error message.

4.3.6 The Edit-Stage-Commit Cycle

Now let's expand our report. Open `report.md` in Zed and add a methodology section:

Listing 4.9 `report.md`

```
# Manufacturing Quality Analysis Report

## Executive Summary

This report analyzes defect rates across three production lines
during Q4 2024.

## Methodology

Data was collected from automated inspection stations at the end of
each production line. Each item was classified as either conforming
or non-conforming based on dimensional tolerances specified in
engineering drawings.

The inspection process captures the following for each item:

- Timestamp of inspection
- Production line identifier
- Pass/fail status
- Specific defect codes for failures
```

Save the file, then stage and commit:

Listing 4.10 terminal

```
git add report.md
git commit -m "Add methodology section describing data collection"
```

Save the file (Ctrl+S). Remember, Git only sees saved changes.

Check what Git sees:

On branch main

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

Listing 4.11 terminal

```
git status
```

```
    modified:   report.md
```

```
no changes added to commit (use "git add" and/or "git commit -a")
```

Git detected that the file changed. To see exactly what changed, use `git diff`:

Listing 4.12 terminal

```
git diff
```

```
diff --git a/report.md b/report.md
```

```
index 8a3c2f1..b5d4e2a 100644
```

```
--- a/report.md
```

```
+++ b/report.md
```

```
@@ -4,3 +4,17 @@
```

```
    This report analyzes defect rates across three production lines  
    during Q4 2024.
```

```
+
```

```
+## Methodology
```

```
+
```

```
+Data was collected from automated inspection stations at the end of  
+each production line. Each item was classified as either conforming
```

```
...
```

Lines starting with + are additions. Lines starting with - would be deletions. This diff view shows exactly what changed, making review straightforward.

In Zed: Click on a modified file in the Source Control panel to see a side-by-side diff view. Additions appear highlighted in green, deletions in red. This visual diff is often easier to read than the terminal output.

Stage and commit the changes:

4.3.7 Comparing to Google Docs

Let's pause and compare what just happened to the Google Docs workflow:

Listing 4.13 terminal

```
git add report.md
git commit -m "Add methodology section describing data collection"
```

In **Google Docs**, you would have opened the document, typed the methodology section, and... that's it. Every keystroke was automatically saved. The version history would show dozens of micro-versions as you typed, backspaced, reworded, and refined.

In **Git**, you wrote and refined the methodology section in Zed (making as many edits as you wanted), saved the file when satisfied, reviewed the changes with `git diff`, staged with `git add`, and committed with a meaningful message.

The Git approach gives you one clean checkpoint with a descriptive message instead of a cluttered history of every keystroke. When you look back at this project's history in six months, you'll see "Add methodology section describing data collection" rather than 47 auto-saved versions showing your drafting process.

For prose documents where you want continuous backup, Google Docs' approach is superior. For code and technical projects where you want meaningful checkpoints of working states, Git's intentional approach wins.

4.4 Viewing Project History

As you make more commits, you'll want to review the project's history. The `git log` command shows the commit history:

Listing 4.14 terminal

```
git log
```

```
commit b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3 (HEAD -> main)
Author: Your Name <your.email@example.com>
Date:   Mon Jan 15 14:30:00 2025 -0500
```

```
    Add methodology section describing data collection
```

```
commit a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0
Author: Your Name <your.email@example.com>
Date:   Mon Jan 15 14:00:00 2025 -0500
```

Start quality analysis report with executive summary

Each commit shows its unique hash, author, date, and message. The HEAD -> main label indicates where you currently are in the history.

For a more compact view:

Listing 4.15 terminal

```
git log --oneline
```

b4c5d6e Add methodology section describing data collection

a1b2c3d Start quality analysis report with executive summary

In Zed: As of writing, Zed does not have a “History” or “Commits” section showing recent commits, but it is in development.

To see what changed in a specific commit:

Listing 4.16 terminal

```
git show b4c5d6e
```

4.5 Undoing Mistakes

Git’s real power emerges when you need to undo mistakes. Let’s explore common scenarios.

4.5.1 Discarding Unsaved Edits (Before Staging)

Suppose you made some changes to `report.md` that you don’t like and haven’t staged yet. You want to go back to the last committed version.

First, **make sure you’ve saved the file**. If your changes only exist in Zed’s buffer and haven’t been saved to disk, simply closing the file without saving (or using Edit → Undo repeatedly) will discard them. Git isn’t involved yet.

If you’ve saved the unwanted changes to disk but haven’t staged them:

This replaces the file’s contents with the last committed version.

Listing 4.17 terminal

```
git restore report.md
```

 Warning

This command permanently discards your changes. Git cannot recover them because they were never committed. Use this carefully.

In Zed: Right-click the file in the Source Control panel and select “Discard Changes.”

4.5.2 Unstaging Changes

Suppose you staged a file but changed your mind. Use `git restore --staged` to move it back to the working directory:

Listing 4.18 terminal

```
git restore --staged report.md
```

The changes remain in your working directory; they’re just no longer staged for the next commit.

In Zed: Uncheck the staging box next to the staged file, or right-click and select “Unstage File.”

4.5.3 Amending the Last Commit

If you made a commit but forgot to include a change or want to fix the commit message:

Listing 4.19 terminal

```
git commit --amend -m "Add methodology section with data collection details"
```

This replaces the previous commit with a new one containing your staged changes and the new message.

4.5.4 Reverting a Commit

To undo a commit that’s already been made while preserving history, use `git revert`:

Listing 4.20 terminal

```
git revert b4c5d6e
```

This creates a new commit that undoes the changes from the specified commit. The original commit remains in the history, but its effects are reversed.

In Zed: There is a reverse arrow underneath the Commit box.

4.6 Branching: Parallel Universes for Your Project

Branches are one of Git’s most powerful features. A branch is an independent line of development that diverges from the main project. You can experiment, add sections, or try a different approach on a branch without affecting the stable main version.

4.6.1 The Google Docs Comparison

In Google Docs, if you wanted to try a completely different structure for your report, you’d have to make a copy of the entire document (“Quality Report - Copy” or “Quality Report v2”). You’d then have two separate documents to manage. If you decided you liked the new structure, you’d have to manually reconcile the two versions or abandon one entirely.

With Git branches, you keep a single project but create parallel timelines. The branch is lightweight and merging branches back together is a first-class operation.

4.6.2 Creating and Switching Branches

Let’s say you want to try adding a data visualization section, but you’re not sure it belongs in this report. Create a branch to experiment:

Listing 4.21 terminal

```
git branch add-visualizations
```

Switch to the new branch:

Listing 4.22 terminal

```
git switch add-visualizations
```

Listing 4.23 terminal

```
git switch -c add-visualizations
```

Or do both in one command:

In Zed: Click the branch name in the bottom-left status bar to see a list of branches. You can switch branches or create new ones from this menu.

Now any commits you make will be recorded on this branch, leaving **main** unchanged.

4.6.3 Working on a Branch

Let's add a visualizations section to our report on this branch. Open **report.md** in Zed and add the new section:

Save the file, then stage and commit:

4.6.4 Viewing Branches

See all branches and which one you're on:

```
* add-visualizations  
  main
```

The asterisk indicates your current branch.

In Zed: The current branch name appears in the bottom-left status bar. Click it to see all branches.

4.6.5 Switching Back and Seeing the Difference

Switch back to main:

Open `report.md` and notice the visualization section is gone. It only exists on the `add-visualizations` branch. Your main branch is exactly as you left it. This is the power of branching: parallel development without interference.

! Save Before Switching Branches

Before switching branches, make sure all your changes are either committed or stashed. If you have unsaved changes in Zed, save them first. If you have uncommitted changes on disk, Git will either carry them to the new branch (if there are no conflicts) or refuse to switch. Get in the habit of committing or stashing your work before switching branches.

4.6.6 Merging Branches

You've decided the visualization section is good. Merge it into main:

```
Updating b4c5d6e..c7d8e9f
Fast-forward
 report.md | 15 +++++
 1 file changed, 15 insertions(+)
```

The branch's commits are now part of main's history. Open `report.md` and the visualization section is there.

4.6.7 Deleting Merged Branches

After merging, you can delete the branch:

```
Deleted branch add-visualizations (was c7d8e9f).
```

4.7 The .gitignore File

Not every file belongs in version control. Large data files, temporary files, and files containing secrets should be excluded. The `.gitignore` file tells Git which files to ignore.

Create a new file in Zed (File → New File), add the following content, and save it as `.gitignore` (note the leading dot) in your project root:

Save the file, then stage and commit:

From now on, Git ignores files matching these patterns. They won't appear in `git status` and can't be accidentally committed.

Finding .gitignore Templates

GitHub maintains a collection of `.gitignore` templates for various languages and frameworks at github.com/github/gitignore. When we start working with Python, we'll use a more comprehensive template.

4.8 Best Practices for Commits

Good commits make your project history useful. Here are guidelines that professional developers follow.

4.8.1 Commit Messages

Write clear, descriptive commit messages. The first line should be a brief summary (50 characters or less). If more detail is needed, add a blank line followed by a longer explanation:

```
Add statistical analysis section with defect correlations
```

```
Include Pareto analysis of defect types showing that three defect  
codes account for 78% of all failures. Add correlation analysis  
between temperature and defect rate.
```

Use imperative mood (“Add section” not “Added section”). Think of the message as completing the sentence “This commit will...”

4.8.2 Commit Size

Make commits that are atomic and focused. Each commit should represent one logical change. If you find yourself writing “and” in your commit message (“Add methodology and fix typo and reorganize introduction”), you should probably make separate commits.

Small, focused commits make it easier to understand history, find when a bug was introduced, and revert specific changes without losing other work.

4.8.3 Commit Frequency

Commit often but meaningfully. A good rule of thumb: commit when you reach a stable state where the document or code is complete and correct. Don’t commit broken work, but also don’t wait until a feature is completely finished.

4.8.4 Save Before Committing

It bears repeating: Git only tracks what’s saved to disk. Develop the habit of saving frequently (Ctrl+S) and always checking `git status` before committing to make sure Git sees what you expect. Look for the dot indicator on Zed’s file tabs, which shows unsaved changes.

4.9 Zed and Git: A Quick Reference

Here’s a summary of how Zed’s GUI features map to Git commands:

Zed Action	Git Command	Location in Zed
View changes	<code>git status</code>	Source Control panel (Ctrl+Shift+G)
Stage file	<code>git add <file></code>	Click checkbox next to file
Unstage file	<code>git restore --staged <file></code>	Unclick checkbox next to staged file
Commit	<code>git commit -m "message"</code>	Enter message, click Commit
View diff	<code>git diff</code>	Click modified file in panel
Discard changes	<code>git restore <file></code>	Right-click → Discard Changes
Switch branch	<code>git switch <branch></code>	Click branch name in status bar
View history	<code>git log</code>	History section in panel

Remember: Zed is calling Git for you. If something doesn’t work in the GUI, try the terminal command to see detailed error messages.

4.10 Summary

In this chapter, you learned the fundamentals of version control with Git. You can now track changes to your projects through intentional commits, understand the three-state model (working directory, staging area, repository), use both terminal commands and Zed's GUI for Git operations, branch and merge to work on features safely, and exclude files with `.gitignore`. These skills form the foundation for collaborative development with GitHub, which we'll explore in Chapter 6.

The key insight is that Git gives you control over your project's history in a way that autosave systems cannot. For prose documents, autosave is often what you want. For code and technical projects, intentional checkpoints with meaningful messages provide a history you can actually use. Start using Git on every project, even personal ones. The discipline of thoughtful commits will serve you throughout your engineering career.

4.11 Glossary

Branch An independent line of development. Branches let you work on features or experiments without affecting the main codebase.

Commit A snapshot of your project at a specific point in time. Each commit has a unique identifier, author, timestamp, and message.

Diff A comparison showing the differences between two versions of a file or set of files.

HEAD A pointer to the current commit you're working from. Usually points to the tip of the current branch.

Merge The process of combining changes from one branch into another.

Repository (Repo) A Git-tracked project, including all files and the complete history of changes stored in the `.git` directory.

Staging Area (Index) A preparation area where you assemble changes before committing them. Gives you control over exactly what goes into each commit.

Working Directory The actual files on your filesystem that you edit. Changes here are not tracked until staged and committed.

Version Control System (VCS) Software that tracks changes to files over time, enabling you to recall specific versions, compare changes, and collaborate with others. Git is a distributed VCS.

.gitignore A file specifying patterns for files that Git should ignore and not track.

4.12 Resources

- [Pro Git Book](#): The definitive free resource on Git, available online

- [Learning Git: A Hands-On and Visual Guide to the Basics of Git](#): Excellent visual introduction
- [Happy Git and GitHub for the useR](#): Practical guide with great troubleshooting advice
- [Git Immersion](#): Hands-on tutorial walking through Git fundamentals
- [Comprehensive Git Tutorial \(4 hours\)](#): Video course covering Git in depth

Listing 4.24 report.md

```
# Manufacturing Quality Analysis Report

## Executive Summary

This report analyzes defect rates across three production lines
during Q4 2024.

## Methodology

Data was collected from automated inspection stations at the end of
each production line. Each item was classified as either conforming
or non-conforming based on dimensional tolerances specified in
engineering drawings.

The inspection process captures the following for each item:

- Timestamp of inspection
- Production line identifier
- Pass/fail status
- Specific defect codes for failures

## Data Visualization

The following charts summarize the key findings:

### Defect Rate by Production Line

Line A showed the lowest defect rate at 2.3%, while Line C had the
highest at 4.7%. Line B fell in the middle at 3.1%.

### Defect Rate Over Time

All three lines showed improvement over the quarter, with the most
significant reduction occurring after the November maintenance cycle.
```

Listing 4.25 terminal

```
git add report.md
git commit -m "Add data visualization section with charts description"
```

Listing 4.26 terminal

```
git branch
```

Listing 4.27 terminal

```
git switch main
```

Listing 4.28 terminal

```
git switch main  
git merge add-visualizations
```

Listing 4.29 terminal

```
git branch -d add-visualizations
```

Listing 4.30 .gitignore

```
# Data files (too large for Git)  
*.csv  
*.xlsx  
data/  
  
# Temporary files  
*.tmp  
*~  
  
# OS-generated files  
.DS_Store  
Thumbs.db  
Desktop.ini  
  
# Editor backup files  
*.bak  
*.swp
```

Listing 4.31 terminal

```
git add .gitignore  
git commit -m "Add .gitignore for data and temporary files"
```

5 Exercises

5.0.1 Question 4.1

What are the three states/areas that files move through in Git?

- A) Draft, Review, Published
- B) Working Directory, Staging Area, Repository
- C) Local, Remote, Cloud
- D) Created, Modified, Deleted

5.0.2 Question 4.2

What does the `git init` command do?

- A) Downloads a repository from GitHub
- B) Creates a new Git repository in the current directory
- C) Commits all changes to the repository
- D) Pushes changes to a remote server

5.0.3 Question 4.3

What does the `git add` command do?

- A) Creates a new file in the project
- B) Moves changes from the working directory to the staging area
- C) Commits changes to the repository
- D) Pushes changes to GitHub

5.0.4 Question 4.4

What is a **commit** in Git?

- A) A promise to finish a feature
- B) A snapshot of your project at a specific point in time
- C) A request to merge branches
- D) A connection to a remote repository

5.0.5 Question 4.5

Why does Git require manual commits instead of autosaving like Google Docs?

- A) Git was created before autosave technology existed
- B) Autosave would use too much disk space
- C) Code requires intentional checkpoints of working states, not every keystroke
- D) Git cannot detect when files change

5.0.6 Question 4.6

What is the purpose of a **branch** in Git?

- A) To permanently delete unwanted files
- B) To create an independent line of development without affecting main
- C) To connect to a remote repository
- D) To compress the repository for storage

5.0.7 Question 4.7

What does the `.gitignore` file do?

- A) Hides the Git repository from other users

- B) Specifies files that Git should not track
- C) Lists all commits in the repository
- D) Encrypts sensitive files in the repository

5.0.8 Question 4.8

True or False: Git can only track changes that have been saved to disk.

- A) True
- B) False

5.0.9 Question 4.9

What command shows the current status of your Git repository, including staged and unstaged changes?

- A) `git log`
- B) `git diff`
- C) `git status`
- D) `git show`

5.0.10 Question 4.10

What does `git commit -m "message"` do?

- A) Sends your changes to GitHub
- B) Creates a snapshot with the provided description
- C) Merges two branches together
- D) Reverts to a previous version

6 Collaboration with GitHub

Git tracks your project’s history on your local machine. GitHub takes this further by hosting your repository in the cloud, enabling collaboration with teammates, providing tools for code review, and serving as a backup for your work. In professional environments, GitHub (or similar platforms) is where engineering teams coordinate their efforts.

This chapter introduces GitHub as a collaboration platform. You’ll learn to synchronize local and remote repositories, contribute to team projects, and use GitHub’s project management features. By the end, you’ll have a complete workflow for contributing to collaborative engineering projects.

6.1 Git is Distributed

Before diving into GitHub, you need to understand a key concept: Git is a *distributed* version control system. Every person who clones a repository gets a complete copy, including the entire history. There is no single “master” copy that everyone depends on. Your local repository is just as complete as anyone else’s.

This distributed nature provides resilience. If GitHub went offline tomorrow, every collaborator would still have the complete project history on their machine. Work can continue offline, and repositories can be synchronized when connectivity returns.

However, distributed systems need a way to coordinate. If ten engineers each have complete repositories on their laptops, how do they share changes? This is where GitHub fits in. It hosts a *remote* repository that serves as a coordination point. Everyone pushes their changes to GitHub and pulls others’ changes from it. GitHub isn’t technically “the source of truth” in Git’s architecture, but teams treat it as a central hub where the official version lives.

6.2 Remote Repositories

A **remote** is a Git repository hosted somewhere other than your local machine. You can have multiple remotes, but most workflows use a single remote called **origin** that points to GitHub.

6.2.1 Cloning an Existing Repository

The most common way to start working with a GitHub repository is to *clone* it. Cloning downloads the complete repository, including all files and history, to your local machine:

Listing 6.1 terminal

```
git clone https://github.com/username/project-name.git
```

This creates a new directory called **project-name** containing the repository. Git automatically configures the remote **origin** to point back to GitHub.

Move into the cloned directory:

Listing 6.2 terminal

```
cd project-name
```

Verify the remote configuration:

Listing 6.3 terminal

```
git remote -v
```

```
origin  https://github.com/username/project-name.git (fetch)
origin  https://github.com/username/project-name.git (push)
```

6.2.2 Connecting a Local Repository to GitHub

If you have an existing local repository (created with `git init` as described in Chapter 4) and want to push it to GitHub, first create a new repository on GitHub (without initializing it with any files). Then connect your local repository:

Verify the connection:

Now your local repository knows where to push changes.

Listing 6.4 terminal

```
git remote add origin https://github.com/username/your-repo.git
```

Listing 6.5 terminal

```
git remote -v
```

6.3 Pushing and Pulling

Synchronizing changes between your local repository and GitHub involves two operations: **push** sends your commits to GitHub, and **pull** downloads commits from GitHub to your machine.

6.3.1 Pushing Changes

After making commits locally, push them to GitHub:

Listing 6.6 terminal

```
git push origin main
```

This command says “push the commits from my local **main** branch to the **origin** remote’s **main** branch.”

The first time you push a new branch, you may need to set up tracking:

The **-u** flag establishes a tracking relationship. After this, you can simply use **git push** without specifying the remote and branch.

6.3.2 Pulling Changes

When collaborators push their commits to GitHub, you need to pull those changes to stay synchronized:

Or simply, if tracking is established:

git pull actually does two things: it *fetches* commits from the remote (downloads them without merging) and then *merges* them into your current branch. If you want to see what’s new on the remote without merging yet:

Listing 6.7 terminal

```
git push -u origin main
```

Listing 6.8 terminal

```
git pull origin main
```

This lets you review incoming changes before integrating them.

6.3.3 Handling Conflicts

When you and a collaborator modify the same lines of a file, Git cannot automatically merge the changes. This is called a **merge conflict**. Git marks the conflicting sections in the file:

To resolve the conflict, edit the file to keep the correct version (or combine both changes), remove the conflict markers, then stage and commit:

Avoiding Conflicts

The best way to handle conflicts is to avoid them. Communicate with your team about who's working on which files. Pull frequently so your local copy stays close to the remote. Make small, focused commits that are easier to merge.

6.4 The Collaborative Workflow

Professional teams follow structured workflows to coordinate contributions. The most common pattern uses feature branches and pull requests.

6.4.1 Feature Branch Workflow

Instead of committing directly to **main**, you create a branch for each feature or fix:

Work on the branch, making commits as you go:

Push the branch to GitHub:

Now your work is backed up on GitHub, but it hasn't affected **main** yet. The next step is to propose merging your branch through a pull request.

Listing 6.9 terminal

```
git pull
```

Listing 6.10 terminal

```
git fetch origin  
git log origin/main --oneline
```

6.4.2 Pull Requests

A **pull request** (PR) is GitHub’s mechanism for proposing that changes from one branch be merged into another. Despite the confusing name (you’re *requesting* that someone *pull* your changes), pull requests are really about review and discussion.

When you push a branch to GitHub, you’ll see a prompt to create a pull request. Click it, or navigate to the repository and click “New pull request.” Select your feature branch as the source and `main` as the destination.

Write a clear title and description explaining what the PR accomplishes, why the changes are needed, and any context reviewers should know. Good PR descriptions save everyone time.

6.4.3 Code Review

Pull requests enable code review, one of the most valuable practices in software engineering. Reviewers can comment on specific lines, suggest changes, ask questions, and ultimately approve or request changes to the PR.

As a contributor, respond to feedback thoughtfully. If you need to make changes, commit them to your branch and push. The PR automatically updates with your new commits.

As a reviewer, focus on correctness, clarity, and maintainability. Ask questions rather than making demands. Remember that code review is a learning opportunity for everyone involved.

6.4.4 Merging Pull Requests

Once the PR is approved and any automated checks pass, you can merge it. GitHub offers several merge strategies. For most workflows, a simple merge commit works well. Click “Merge pull request” on GitHub.

After merging, delete the feature branch. It’s served its purpose, and keeping old branches around creates clutter:

Listing 6.11 `analyze_sensors.py`

```
def process_reading(value):
#<<<<<<< HEAD
    return value * 1.05 # Your change: 5% calibration factor
#=====
    return value * 1.03 # Their change: 3% calibration factor
#>>>>>>> origin/main
```

Listing 6.12 terminal

```
git add analyze_sensors.py
git commit -m "Resolve calibration factor conflict, use 5%"
```

6.5 Project Management on GitHub

GitHub provides tools beyond code hosting that help teams coordinate work. These features turn GitHub into a project management platform.

6.5.1 Issues

Issues track tasks, bugs, feature requests, and discussions. Create an issue for anything that needs to be done or decided. Good issues include a clear title, a detailed description of the problem or desired feature, relevant context like error messages or specifications, and labels to categorize the issue.

Reference issues in your commits and pull requests using the `#` notation:

GitHub automatically links the commit to issue `#42` and closes the issue when the commit reaches `main`.

6.5.2 Labels and Milestones

Labels categorize issues: `bug`, `enhancement`, `documentation`, `help wanted`. Create labels that match your team's workflow.

Milestones group issues into releases or sprints. Assign issues to a milestone to track progress toward a goal, like “v1.0 Release” or “Sprint 3”.

Listing 6.13 terminal

```
git switch -c feature/add-anomaly-detection
```

Listing 6.14 terminal

```
git add anomaly.py
git commit -m "Implement statistical anomaly detection"
git add tests/test_anomaly.py
git commit -m "Add tests for anomaly detection"
```

6.5.3 GitHub Projects

Projects provide Kanban-style boards for visualizing work. Create columns like “To Do,” “In Progress,” and “Done.” Add issues and pull requests to the board and move them between columns as work progresses.

Projects help teams see the big picture: what’s being worked on, what’s blocked, and what’s completed.

6.5.4 Discussions

Discussions provide a forum for conversations that aren’t tied to specific code changes. Use discussions for questions, ideas, announcements, and community engagement.

6.6 Forking: Contributing to Others’ Projects

When you want to contribute to a project you don’t have write access to (common with open-source projects), you use the **fork** workflow.

A **fork** is your personal copy of someone else’s repository. You can freely experiment with changes without affecting the original project.

The fork workflow is as follows. First, click “Fork” on the original repository’s GitHub page. This creates a copy under your account. Clone your fork locally:

Add the original repository as an “upstream” remote so you can pull in updates:

Create a branch, make changes, and push to your fork:

Listing 6.15 terminal

```
git push -u origin feature/add-anomaly-detection
```

Listing 6.16 terminal

```
git switch main  
git pull  
git branch -d feature/add-anomaly-detection
```

On GitHub, create a pull request from your fork's branch to the original repository's main branch. The maintainers will review your contribution.

To keep your fork synchronized with the original:

6.7 GitHub CLI

The GitHub CLI (`gh`) lets you interact with GitHub from your terminal. This is faster than switching to a browser for common operations.

Check your authentication status:

Create a new repository:

Create a pull request:

List open pull requests:

Check out a pull request locally for testing:

View issues:

The CLI integrates Git and GitHub operations, reducing context switching between terminal and browser.

6.8 Repository Organization

Well-organized repositories are easier to understand and contribute to. Follow these conventions to make your projects professional.

Listing 6.17 terminal

```
git commit -m "Fix temperature sensor parsing, closes #42"
```

Listing 6.18 terminal

```
git clone https://github.com/your-username/project-name.git
```

6.8.1 README Files

Every repository should have a `README.md` file at the root. This is the first thing visitors see. Include the project name and brief description, installation instructions, usage examples, contribution guidelines, and license information.

6.8.2 License

Add a `LICENSE` file to clarify how others can use your code. For open-source projects, MIT and Apache 2.0 are common choices. For proprietary work, specify that all rights are reserved.

6.8.3 Contributing Guidelines

For collaborative projects, add a `CONTRIBUTING.md` file explaining how to set up a development environment, coding standards and style guidelines, how to run tests, and the pull request process.

6.8.4 Directory Structure

Organize code logically. A typical Python project structure might be:

```
project-name/  
  README.md  
  LICENSE  
  pyproject.toml  
  src/  
    project_name/  
      __init__.py  
      main.py  
  tests/
```

Listing 6.19 terminal

```
git remote add upstream https://github.com/original-owner/project-name.git
```

Listing 6.20 terminal

```
git switch -c fix/typo-in-readme
git commit -am "Fix typo in README"
git push origin fix/typo-in-readme
```

```
test_main.py
docs/
  index.md
```

6.9 Security Considerations

Never commit secrets to Git. This includes API keys, passwords, database credentials, private keys, and tokens. Once a secret is in Git history, it's extremely difficult to remove completely.

Use environment variables or dedicated secret management tools for sensitive configuration. Add files containing secrets to `.gitignore` before you accidentally commit them.

If you do accidentally commit a secret, immediately rotate it (generate a new key) rather than trying to remove it from history. Assume the secret is compromised.

GitHub offers features to help with security. Enable Dependabot to receive alerts about vulnerabilities in your dependencies. Use branch protection rules to require reviews before merging. Consider signing your commits with GPG keys to verify authorship.

6.10 Summary

In this chapter, you learned how GitHub enables collaborative development. You can now synchronize local and remote repositories using push and pull, collaborate using the feature branch workflow and pull requests, participate in code review, use GitHub's project management features (issues, projects, milestones), contribute to projects you don't own using forks, and work efficiently with the GitHub CLI.

The combination of Git and GitHub forms the backbone of modern software development. These tools and workflows apply whether you're working on a two-person team or contributing

Listing 6.21 terminal

```
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```

Listing 6.22 terminal

```
gh auth status
```

to projects with thousands of collaborators. Practice these patterns on every project, and they'll become second nature.

6.11 Glossary

Clone Creating a local copy of a remote repository, including all files and history.

Fork A personal copy of someone else's repository on GitHub. Allows you to experiment freely and propose changes via pull requests.

Merge Conflict Occurs when Git cannot automatically combine changes because the same lines were modified differently. Requires manual resolution.

Origin The conventional name for your primary remote repository, typically pointing to GitHub.

Pull Downloading commits from a remote repository and merging them into your local branch.

Pull Request (PR) A GitHub feature for proposing that changes from one branch be merged into another. Enables code review and discussion.

Push Uploading your local commits to a remote repository.

Remote A Git repository hosted somewhere other than your local machine, such as on GitHub.

Upstream In the fork workflow, the original repository that you forked from.

Issue A GitHub feature for tracking tasks, bugs, feature requests, and discussions related to a repository.

Code Review The practice of examining code changes before they're merged, checking for bugs, style, and clarity.

6.12 Resources

- [GitHub Documentation](#): Official comprehensive documentation
- [GitHub Skills](#): Interactive courses for learning GitHub
- [GitHub Flow Guide](#): Explanation of the recommended workflow

Listing 6.23 terminal

```
gh repo create my-new-project --public --source=. --remote=origin
```

Listing 6.24 terminal

```
gh pr create --title "Add anomaly detection" --body "Implements statistical anomaly detection"
```

- [Understanding the GitHub Flow](#): Visual guide to branches and pull requests
- [GitHub CLI Manual](#): Complete reference for the `gh` command
- [Open Source Guides](#): Best practices for open source projects

6.13 Exercises

6.13.1 Question 5.1

What is the relationship between Git and GitHub?

- A) They are the same software
- B) Git is a version control system; GitHub is a hosting platform for Git repositories
- C) GitHub is a version control system; Git is a hosting platform
- D) Git only works when connected to GitHub

6.13.2 Question 5.2

What does `git clone` do?

- A) Creates a new empty repository
- B) Downloads a complete copy of a remote repository
- C) Uploads local changes to GitHub
- D) Creates a branch of the current repository

Listing 6.25 terminal

```
gh pr list
```

Listing 6.26 terminal

```
gh pr checkout 123
```

6.13.3 Question 5.3

What does `git push` do?

- A) Downloads changes from a remote repository
- B) Uploads local commits to a remote repository
- C) Creates a new branch
- D) Merges two branches together

6.13.4 Question 5.4

What does `git pull` do?

- A) Uploads local commits to a remote repository
- B) Downloads commits from a remote repository and merges them
- C) Creates a pull request on GitHub
- D) Deletes a remote branch

6.13.5 Question 5.5

What is a **pull request** on GitHub?

- A) A request to download a repository
- B) A proposal to merge changes from one branch into another

Listing 6.27 terminal

```
gh issue list  
gh issue view 42
```

- C) A request to delete a repository
- D) A request for more storage space

6.13.6 Question 5.6

What is a **merge conflict**?

- A) When two repositories cannot connect to each other
- B) When Git cannot automatically combine changes because the same lines were modified differently
- C) When a branch is deleted accidentally
- D) When GitHub servers are down

6.13.7 Question 5.7

What is a **fork** on GitHub?

- A) A tool for splitting large files
- B) A personal copy of someone else's repository under your account
- C) A type of merge strategy
- D) A way to delete repositories

6.13.8 Question 5.8

What is the conventional name for the primary remote repository?

- A) main

- B) master
- C) origin
- D) upstream

6.13.9 Question 5.9

Why should you NEVER commit secrets (like API keys or passwords) to Git?

- A) Git cannot store special characters
- B) Secrets make the repository too large
- C) Once in Git history, secrets are extremely difficult to remove completely
- D) GitHub automatically rejects commits with secrets

6.13.10 Question 5.10

What is the purpose of **code review** in the pull request workflow?

- A) To automatically fix bugs in the code
- B) To examine changes for bugs, style, and clarity before merging
- C) To compress the code for faster performance
- D) To convert code between programming languages

A Homebrew

A.1 What Problem Does Homebrew Solve?

When you need a new app on your phone, you open the App Store, search for what you want, and tap “Install.” The App Store handles everything behind the scenes: downloading the app, putting it in the right place, and making sure it works with your device. You never think about *where* the app lives on your phone or *how* it got there.

Now imagine you need to install a programming tool that doesn’t exist in the App Store. Without a package manager, you’d have to visit a website, find the correct download link for your specific computer, extract compressed files, figure out where to put them, and manually configure your system to recognize the new tool. If that tool depends on *other* tools to work, you’d need to track those down and install them first. It’s tedious, error-prone, and frustrating.

Homebrew is essentially the App Store for command-line tools and developer software. It gives your Mac the same convenient installation experience that Linux users have enjoyed for decades. You tell Homebrew what you want, and it handles the rest: downloading, installing, resolving dependencies, and configuring paths.

A.2 Why This Matters for Your Workflow

Throughout this book, you’ll need various tools installed on your computer: version control systems, programming language runtimes, database utilities, and more. Without Homebrew, each installation becomes its own adventure of Googling instructions and hoping nothing breaks. With Homebrew, installing new tools becomes predictable and consistent.

Homebrew also makes your setup *reproducible*. If you get a new computer or need to help a classmate configure their machine, you can simply share a list of packages rather than walking through dozens of manual installation steps.

A.3 Installation Instructions

Open the **Terminal** application on your Mac. You can find it by pressing **Cmd + Space** to open Spotlight Search, then typing “Terminal” and pressing Enter.

Once Terminal is open, copy and paste the following command, then press Enter:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

The installer will explain what it plans to do and ask for your password (the same password you use to log into your Mac). As you type your password, you won’t see any characters appear on screen—this is normal security behavior. Just type your password and press Enter.

The installation takes a few minutes. When it finishes, **read the output carefully**. Homebrew will print instructions telling you to run one or two additional commands to complete the setup. These commands add Homebrew to your system’s PATH, which is how your computer knows where to find programs. Copy and run those commands exactly as shown.

To verify everything worked, close your Terminal window, open a fresh one, and type **brew --version**. If you see a version number, you’re all set.

A.4 What Happens Next

Once Homebrew is installed, you’ll have access to thousands of tools through simple commands. We’ll introduce specific commands as we need them throughout the book. For now, just know that Homebrew is the foundation that makes installing everything else straightforward.

If you’re curious, Homebrew’s website at brew.sh has documentation and a searchable list of available packages. But don’t feel pressured to explore on your own yet—we’ll guide you through each tool as it becomes relevant to your work.

B Git Configuration

Before you can use Git effectively, you need to configure it with your identity and preferences. Git configurations are settings stored in configuration files that customize how Git behaves. This appendix walks you through the essential setup and common configuration options.

B.1 Configuration Levels

Git stores configurations at three levels, each overriding the previous:

1. **System** (`/etc/gitconfig`): Applies to all users on the machine. Rarely modified unless you're a system administrator.
2. **Global** (`~/.gitconfig` or `~/.config/git/config`): Applies to all repositories for the current user. This is where you set your identity and preferences.
3. **Local** (`.git/config` in a repository): Applies only to that specific repository. Useful for project-specific settings, like using a work email for work projects.

When you run `git config`, you specify which level to modify with `--system`, `--global`, or `--local` (the default).

B.2 Checking Your Configuration

To see your current configuration, including where each setting comes from:

Listing B.1 terminal

```
git config --list --show-origin
```

This displays all active settings and which configuration file defines each one.

To check a specific setting:

Listing B.2 terminal

```
git config user.name
git config user.email
```

B.3 Required: Your Identity

Git requires your name and email address. Every commit you make is stamped with this information, creating an audit trail of who made which changes. This isn't about authentication, but about attribution within the repository history.

Set your identity globally:

Listing B.3 terminal

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

These values should match how you want to appear in commit logs. For professional work, use your full name. The email address will be visible to anyone who can see the repository, so choose one you're comfortable sharing publicly.

Using Different Identities

If you use Git for both personal and work projects, you might want different identities. Set your personal identity globally, then override it for work repositories:

Listing B.4 terminal

```
cd ~/work/company-project
git config --local user.email "your.name@company.com"
```

Only this repository will use the work email; others will use your global setting.

B.4 Verifying Installation

Before configuring, confirm Git is installed and check the version:

Listing B.5 terminal

```
git --version
```

You should see output like `git version 2.43.0`. If you get a “command not found” error, you need to install Git.

B.4.1 Installing Git

On **Windows**, the recommended approach is using WinGet:

Listing B.6 terminal

```
winget install Git.Git
```

After installation, close and reopen your terminal for the `git` command to be available.

On **macOS**, install via Homebrew:

Listing B.7 terminal

```
brew install git
```

After installation, verify with `git --version` and ensure you have version 2.28 or higher.

B.5 Default Branch Name

Historically, Git’s default branch was called `master`. The community has largely moved to `main` as a more neutral default. Configure Git to use `main` for new repositories:

This only affects repositories you create with `git init` going forward. Existing repositories keep their current branch names.

B.6 Line Endings

Different operating systems use different characters to represent line endings. Windows uses CRLF (carriage return + line feed), while macOS and Linux use LF (line feed only). This can cause problems when collaborators use different systems.

Listing B.8 terminal

```
git config --global init.defaultBranch main
```

Configure Git to handle this automatically:

On **Windows**:

Listing B.9 terminal

```
git config --global core.autocrlf true
```

This converts **LF** to **CRLF** when checking out files and converts back to **LF** when committing.

On **macOS**:

Listing B.10 terminal

```
git config --global core.autocrlf input
```

This keeps **LF** endings in the repository and only converts **CRLF** to **LF** on commit (in case Windows files sneak in).

B.7 Default Editor

When Git needs you to write a commit message or resolve a conflict, it opens a text editor. By default, this is often **vim**, which can be confusing for beginners.

Set your preferred editor:

The **--wait** flag tells Git to wait until you close the editor before continuing.

B.8 Useful Aliases

Git aliases create shortcuts for common commands. They save typing and can make Git more intuitive:

Now you can use **git s** instead of **git status --short**, and so on.

Listing B.11 terminal

```
# For Zed
git config --global core.editor "zed --wait"
```

Listing B.12 terminal

```
# Short status
git config --global alias.s "status --short"

# Compact log with graph
git config --global alias.lg "log --oneline --graph --all"

# Amend the last commit without editing the message
git config --global alias.amend "commit --amend --no-edit"

# Undo the last commit but keep changes staged
git config --global alias.undo "reset --soft HEAD~1"

# Show what you're about to push
git config --global alias.unpushed "log @{u}.. --oneline"
```

B.9 Modern Authentication

For GitHub specifically, using the GitHub CLI (`gh auth login`) or SSH keys is more secure and convenient than HTTPS credentials. See [Appendix C](#) for details.

B.10 Viewing Your Configuration File

Your global configuration is stored in a plain text file. You can view and edit it directly:

Listing B.13 terminal

```
git config --global --edit
```

This opens your configuration file in your default editor. A typical configuration looks like:
You can edit this file directly instead of using `git config` commands.

Listing B.14 ~/.gitconfig

```
[user]
  name = Your Name
  email = your.email@example.com
[init]
  defaultBranch = main
[core]
  editor = code --wait
  autocrlf = input
[alias]
  s = status --short
  lg = log --oneline --graph --all
[pull]
  rebase = false
[credential]
  helper = osxkeychain
```

B.11 Recommended Starter Configuration

Here's a complete configuration script to run after installing Git. Customize the name and email:

B.12 Summary

Git configuration establishes your identity and customizes Git's behavior. You learned how Git stores configurations at system, global, and local levels, how to set the required identity settings (`user.name` and `user.email`), how to configure defaults for branch names, line endings, and editors, and how to create aliases for common commands. With these settings in place, Git will work smoothly and your commits will be properly attributed.

B.13 Glossary

Configuration Level The scope at which a Git setting applies, either system (all users), global (current user), or local (current repository).

Line Endings Characters that mark the end of a line in text files. Windows uses CRLF, while macOS/Linux use LF.

Alias A shortcut name for a Git command or sequence of commands.

Listing B.15 terminal

```
# Identity
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"

# Modern defaults
git config --global init.defaultBranch main
git config --global core.autocrlf input # Use 'true' on Windows

# Editor (choose one)
git config --global core.editor "zed --wait"

# Helpful aliases
git config --global alias.s "status --short"
git config --global alias.lg "log --oneline --graph --all"
git config --global alias.amend "commit --amend --no-edit"

# Verify
git config --list
```

Credential Helper A Git feature that caches or stores authentication credentials so you don't have to enter them repeatedly.

B.14 Resources

- [First-Time Git Setup](#) - Official Git documentation
- [Customizing Git Configuration](#) - Deep dive into all configuration options
- [Set Up Git](#) - GitHub's setup guide
- [Setting Your Username in Git](#) - GitHub's identity configuration guide
- [Pro Git Book](#) - Free comprehensive Git reference

C GitHub Configuration

GitHub hosts remote repositories and provides collaboration features, but your local Git installation needs to authenticate with GitHub to push and pull. This appendix covers creating a GitHub account, authenticating securely, and setting up the GitHub CLI for streamlined workflows.

C.1 Creating a GitHub Account

If you don't have a GitHub account, create one at github.com/signup. Choose a professional username since it becomes part of your repository URLs and identifies you in contributions to open-source projects.

When selecting a username, consider using your real name or a professional handle, avoid numbers or characters that make the name hard to remember, and keep it consistent with your other professional accounts.

After creating your account, verify your email address by clicking the link GitHub sends. This is required for many GitHub features to work properly.

C.2 Authentication Methods

Git needs to prove your identity when pushing to or pulling from private repositories on GitHub. There are two main authentication methods: HTTPS with the GitHub CLI, and SSH keys.

C.2.1 Recommended: GitHub CLI Authentication

The GitHub CLI (`gh`) is the simplest way to authenticate. It handles credential storage automatically and integrates Git operations with GitHub features.

Install the GitHub CLI:

On **Windows** with WinGet:

On **macOS** with Homebrew:

Listing C.1 terminal

```
winget install GitHub.cli
```

Listing C.2 terminal

```
brew install gh
```

After installation, close and reopen your terminal, then verify:

Authenticate with GitHub:

The CLI walks you through the authentication process interactively. When prompted, select GitHub.com (unless you use GitHub Enterprise). Choose HTTPS as the preferred protocol for Git operations. When asked about authenticating Git with your GitHub credentials, select Yes. Select Login with a web browser (this is the most secure option).

The CLI generates a one-time code and opens your browser. Log in to GitHub if needed, enter the code, and authorize the GitHub CLI. Back in your terminal, you'll see confirmation that authentication succeeded.

Verify your authentication:

```
github.com
  Logged in to github.com as your-username (oauth_token)
  Git operations for github.com configured to use https protocol.
  Token: gh_*****
```

Now Git operations to GitHub will authenticate automatically.

C.3 Configuring Your GitHub Profile

Your GitHub profile represents you professionally. Take time to complete it.

Navigate to Settings → Profile and add a professional profile photo (your face, not a logo or meme), a bio describing your role and interests, your location and organization (if appropriate), and links to your website or portfolio.

The profile README is a special feature: create a repository with the same name as your username (e.g., `username/username`), and its README appears on your profile page. This is a good place to highlight your skills and projects.

Listing C.3 terminal

```
gh --version
```

Listing C.4 terminal

```
gh auth login
```

C.4 GitHub CLI Configuration

The GitHub CLI has configuration options beyond authentication.

View current configuration:

Set your default editor:

Set the preferred Git protocol:

Set the default browser:

C.5 Essential GitHub CLI Commands

Once authenticated, the GitHub CLI streamlines many workflows. These are the commands you'll use most frequently.

Create a repository:

Clone a repository:

View repository in browser:

Create a pull request:

List and view pull requests:

Check out a pull request locally:

Create an issue:

List and view issues:

View your notifications:

Listing C.5 terminal

```
gh auth status
```

Listing C.6 terminal

```
gh config list
```

C.6 Notifications and Watching

GitHub sends notifications for activity on repositories you watch. Managing notifications prevents inbox overload.

Access notification settings at Settings → Notifications. Configure whether to receive email notifications, participating notifications (when you're mentioned or assigned), and watching notifications (all activity on watched repos).

For repositories, choose your watch level by visiting the repository and clicking “Watch.” Options include participating only (you're mentioned or assigned), all activity, ignore, or custom selections.

C.7 Connecting Git and GitHub

Ensure your Git identity matches your GitHub account:

If your Git email doesn't match any GitHub email, commits won't be linked to your GitHub account. Either add the email to GitHub or change your Git configuration.

C.8 Troubleshooting Authentication

“Permission denied” when pushing:

Verify you're authenticated:

“Repository not found” error:

This usually means you don't have access. Verify the repository exists and you have permission. Check for typos in the URL. If it's a private repository, ensure you're authenticated.

Credential issues:

Listing C.7 terminal

```
gh config set editor "zed --wait"
```

Listing C.8 terminal

```
gh config set git_protocol https
# or
gh config set git_protocol ssh
```

If Git asks for credentials repeatedly, your credential helper may not be configured. See Appendix B for credential helper setup, or use `gh auth setup-git` to configure Git to use GitHub CLI authentication.

C.9 Summary

GitHub authentication connects your local Git to the remote platform. You learned how to create and configure a GitHub account, authenticate using the GitHub CLI (recommended) or SSH keys, configure the GitHub CLI for your workflow, and use essential CLI commands for repositories, pull requests, and issues. With authentication configured, you can seamlessly push your work to GitHub and collaborate with others.

C.10 Glossary

GitHub CLI (gh) A command-line tool for interacting with GitHub, including authentication, repository management, pull requests, and issues.

HTTPS Authentication Authentication using tokens or credentials over the HTTPS protocol. The GitHub CLI manages this automatically.

C.11 Resources

- [GitHub CLI Documentation](#) - Complete reference for `gh` commands
- [GitHub CLI Installation](#) - Installation instructions for all platforms
- [Creating a GitHub Account](#) - Official account setup guide
- [About GitHub and Git](#) - Understanding the relationship between Git and GitHub

Listing C.9 terminal

```
gh config set browser "chrome"
```

Listing C.10 terminal

```
# Create a new repository and push current directory
gh repo create my-project --public --source=. --remote=origin --push

# Create interactively
gh repo create
```

Listing C.11 terminal

```
gh repo clone username/repo-name
```

Listing C.12 terminal

```
gh repo view --web
```

Listing C.13 terminal

```
gh pr create --title "Add feature X" --body "Description of changes"

# Create interactively
gh pr create
```

Listing C.14 terminal

```
gh pr list
gh pr view 42
gh pr view 42 --web # Open in browser
```

Listing C.15 terminal

```
gh pr checkout 42
```

Listing C.16 terminal

```
gh issue create --title "Bug in parser" --body "Steps to reproduce..."
```

Listing C.17 terminal

```
gh issue list  
gh issue view 123
```

Listing C.18 terminal

```
gh status
```

Listing C.19 terminal

```
# Check your Git email  
git config --global user.email  
  
# This should match one of your GitHub emails  
# Verify at: https://github.com/settings/emails
```

Listing C.20 terminal

```
gh auth status
```

D WinGet

D.1 What Problem Does WinGet Solve?

When you need a new app on your phone, you open the App Store, search for what you want, and tap “Install.” The store handles everything behind the scenes: downloading the app, putting it in the right place, and making sure it works with your device. You never think about *where* the app lives or *how* it got there.

Now imagine you need to install a programming tool that doesn’t exist in the Microsoft Store. Without a package manager, you’d visit a website, hunt for the correct download link, run an installer, click through license agreements, choose installation options you may not understand, and hope the tool ends up somewhere your system can find it. If that tool depends on *other* tools to work, you’d need to track those down and install them first. It’s tedious, error-prone, and frustrating.

WinGet is Microsoft’s answer to this problem. It’s a command-line package manager built into modern versions of Windows that gives you the same convenient installation experience that Linux and Mac users have enjoyed for years. You tell WinGet what you want, and it handles the rest: downloading, installing, resolving dependencies, and configuring paths.

D.2 Why This Matters for Your Workflow

Throughout this book, you’ll need various tools installed on your computer: version control systems, programming language runtimes, database utilities, and more. Without WinGet, each installation becomes its own adventure of Googling instructions and navigating different installer interfaces. With WinGet, installing new tools becomes predictable and consistent.

WinGet also makes your setup *reproducible*. If you get a new computer or need to help a classmate configure their machine, you can share a list of packages rather than walking through dozens of manual installation steps.

D.3 Checking If WinGet Is Already Installed

WinGet comes pre-installed on Windows 11 and recent versions of Windows 10. To check if you already have it, open **PowerShell** or **Command Prompt** by pressing **Win + X** and selecting “Terminal” (on Windows 11) or by searching for “PowerShell” in the Start menu.

Once your terminal is open, type `winget --version` and press Enter. If you see a version number, you’re all set and can skip to the next section. If you see an error message saying the command isn’t recognized, you’ll need to install it.

D.4 Installation Instructions (If Needed)

The easiest way to get WinGet is through the Microsoft Store. Search for “App Installer” in the Microsoft Store and install it (or update it if it’s already present). The App Installer package includes WinGet as a component.

Alternatively, you can download the latest release directly from GitHub at <https://github.com/microsoft/winget-cli/releases>. Look for the file ending in `.msixbundle`, download it, and double-click to install.

After installation, close any open terminal windows and open a fresh one. Type `winget --version` again to confirm everything is working. If you see a version number, you’re ready to go.

D.5 A Note on Permissions

Some software installations require administrator privileges. When you run a WinGet command that needs elevated permissions, Windows will display a User Account Control (UAC) prompt asking you to confirm. This is normal and expected—just click “Yes” to allow the installation to proceed.

D.6 What Happens Next

Once WinGet is available, you’ll have access to thousands of tools through simple commands. We’ll introduce specific commands as we need them throughout this book. For now, just know that WinGet is the foundation that makes installing everything else straightforward.

If you’re curious, Microsoft’s documentation at learn.microsoft.com/en-us/windows/package-manager has more details and a searchable repository of available packages. But don’t feel

pressured to explore on your own yet, we'll guide you through each tool as it becomes relevant to your work.

E Zed (WIP)